

**CONCEITOS-
-CHAVE**

acoplamento	268
coesão	266
componentes	
adaptação	281
classificação	282
composição	281
orientados a	
objetos.	260
qualificação	280
tradicionais.	275
WebApp	274
desenvolvimento	
baseado em	
componentes.	279
diretrizes de projeto.	265
engenharia de	
domínio	280
notação tabular	
de projetos.	277
projeto de conteúdo	274

O projeto de componentes ocorre depois de a primeira iteração do projeto da arquitetura tiver sido completa. Neste estágio, a estrutura geral dos dados e programas do software já foi estabelecida. O intuito é traduzir o modelo de projeto em software operacional. Porém, o nível de abstração do modelo de projetos existente é relativamente alto e o nível de abstração do programa operacional é baixo. A tradução pode ser desafiadora, é uma porta aberta para a introdução de erros sutis difíceis de detectar e corrigir em estágios posteriores do processo de software. Em uma famosa palestra, Edsger Dijkstra, um importante contribuidor para nosso entendimento de projeto de software, afirmou [Dij72]:

Software parece ser diferente de muitos outros produtos, em que, via de regra, maior qualidade implica preços mais elevados. Aqueles que realmente querem software confiável descobrirão que devem, primeiro, encontrar um meio de evitar a maioria dos bugs e, como resultado, o processo de programação se tornará mais barato... Programadores eficientes... Não devem perder tempo depurando — e, para início de conversa, não devem introduzir erros.

Embora essas palavras tenham sido proferidas muitos anos atrás, ainda são verdadeiras hoje em dia. Ao traduzirmos o modelo de projetos em código-fonte, devemos seguir um conjunto de princípios de projeto que não apenas realizam a tradução, como também não “introduzam bugs desde o início”.

PANORAMA

O que é? Um conjunto completo de componentes de software é definido durante o projeto da arquitetura. Porém, os detalhes de processamento e estruturas de dados internas de cada componente não são representados em um nível de abstração próximo ao código. O projeto de componentes define as estruturas de dados, os algoritmos, as características das interfaces e os mecanismos de comunicação alocados a cada componente de software.

Quem realiza? Um engenheiro de software realiza o projeto de componentes.

Por que é importante? Você deve ser capaz de determinar se o software irá funcionar ou não antes de construí-lo. O projeto de componentes representa o software de maneira que lhe permita revisar os detalhes do projeto em termos de correção e consistência com outras representações de projeto (isto é, os projetos de interfaces, arquitetura e dados). Ele fornece um meio para avaliar se as estruturas de dados, interfaces e algoritmos vão funcionar.

Quais são as etapas envolvidas? As representações de projeto de dados, arquitetura e interfaces formam a base para o projeto de com-

ponentes. A definição de classes ou a narrativa de processamento para cada um dos componentes é traduzida em um projeto detalhado que faz uso de formas esquemáticas ou baseadas em texto que especificam estruturas de dados internas, detalhes de interfaces locais e lógica de processamento. A notação de projeto engloba diagramas UML e representações complementares. O projeto procedural é especificado usando-se um conjunto de construções da programação estruturada. Normalmente é possível adquirir-se componentes de software reutilizáveis em vez de construir novos componentes.

Qual é o artefato? O projeto para cada componente, representado em notação gráfica, tabular ou baseada em texto, é o principal artefato durante o projeto de componentes.

Como garantir que o trabalho foi realizado corretamente? Uma revisão do projeto é realizada. O projeto é examinado para determinar se as estruturas de dados, interfaces, sequências de processamento e condições lógicas estão corretas e irão produzir a transformação de controle ou de dados apropriada atribuída ao componente durante etapas anteriores do projeto.

É possível representar o projeto de componentes usando uma linguagem de programação. Em essência, o programa é criado por meio do modelo de projeto da arquitetura como diretriz. Uma abordagem alternativa é representar o projeto de componentes usando alguma representação intermediária (por exemplo, gráfica, tabular ou baseada em texto) que possa ser traduzida facilmente em código-fonte. Independentemente do mecanismo utilizado para representar o projeto de componentes, as estruturas de dados, as interfaces e os algoritmos definidos devem atender uma série de diretrizes de projeto bem fundamentadas que auxiliam a evitar erros à medida que o projeto procedural evolui. Neste capítulo, examinaremos essas diretrizes de projeto e os métodos disponíveis para atingi-las.

10.1 O QUE É COMPONENTE?

“Detalhes não são detalhes. Eles fazem o projeto.”

Charles Eames

Componente é um bloco construtivo modular para software de computador. Mais formalmente, a Especificação da Linguagem de Modelagem Unificada da OMG (*OMG Unified Modeling Language Specification* [OMG03a]) define componente como “... Uma parte modular, possível de ser implantada e substituível de um sistema que encapsula implementação e expõe um conjunto de interfaces”.

Conforme discutido no Capítulo 9, os componentes preenchem a arquitetura de software e, como consequência, desempenham um papel para alcançar os objetivos e requisitos do sistema a ser construído. Pelo fato de os componentes residirem na arquitetura de software, devem se comunicar e colaborar com outros componentes e entidades (por exemplo, outros sistemas, dispositivos, pessoas) existentes fora dos limites do software.

O verdadeiro significado do termo *componente* diferirá dependendo do ponto de vista do engenheiro de software que o utiliza. Nas seções seguintes, examinaremos três importantes visões do que é um componente e como é utilizado à medida que a modelagem de projetos prossegue.

10.1.1 Uma visão orientada a objetos

PONTO-CHAVE

Sob o ponto de vista orientado a objetos, um componente é um conjunto de classes colaborativas.

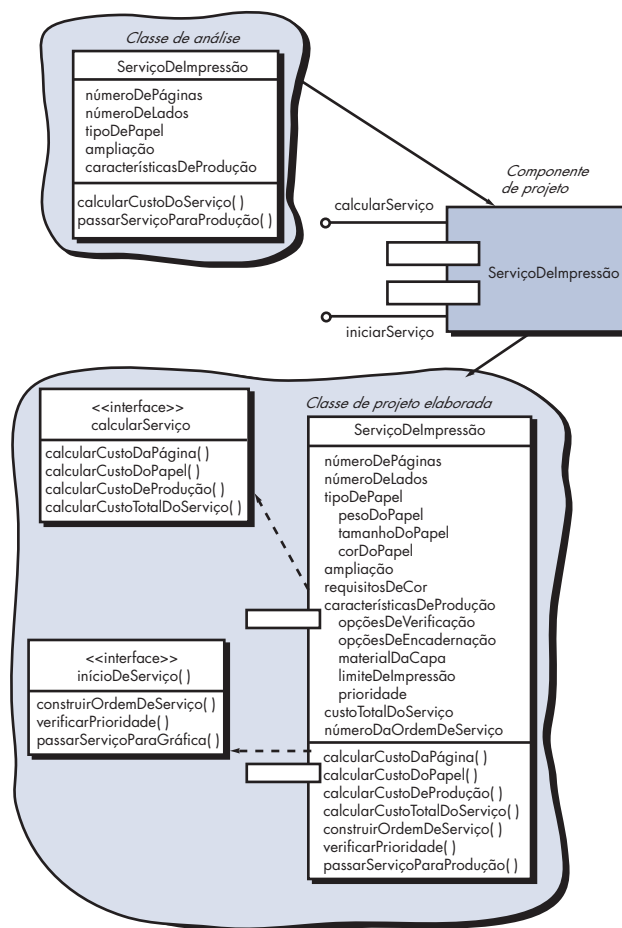
No contexto da engenharia de software orientada a objetos, um componente contém um conjunto de classes colaborativas.¹ Cada classe contida em um componente foi completamente elaborada para incluir todos os atributos e operações relevantes à sua implementação. Como parte da elaboração do projeto, todas as interfaces que permitem que as classes se comuniquem e colaborem com outras classes de projeto também precisam ser definidas. Para tanto, começamos com o modelo de requisitos e elaboramos as classes de análise (para componentes que se relacionam com o domínio do problema), bem como as classes de infraestrutura (para componentes que oferecem suporte a serviços para o domínio do problema).

Para ilustrarmos o processo de elaboração de projeto, consideremos o software a ser criado para uma sofisticada gráfica. O objetivo geral do software é coletar os requisitos do cliente na recepção da loja, orçar um trabalho e, em seguida, passar a tarefa para um centro de produção automatizado. Durante a engenharia de requisitos, foi obtida uma classe de análise denominada **ServiçoDeImpressão**. Os atributos e operações definidos durante a análise são indicados na parte superior da Figura 10.1. Durante o projeto da arquitetura, **ServiçoDeImpressão** é definida como um componente na arquitetura de software e é representada usando notação abreviada² UML no centro à direita da figura. Observe que **ServiçoDeImpressão** possui duas interfaces, *calcularServiço*, que fornece capacidade para orçar um trabalho e *iniciarServiço*, que passa adiante a tarefa para o centro de produção. Estas são representadas usando-se os símbolos de “pirulito” exibidos à esquerda do retângulo do componente.

¹ Em alguns casos, um componente pode conter uma única classe.

² Os leitores que não estiverem familiarizados com a notação UML devem consultar o Apêndice 1.

FIGURA 10.1

Elaboração de um componente de projeto

Lembre-se de que tanto a modelagem da análise quanto a modelagem do projeto são ações iterativas. Elaborar a classe de análise original poderia exigir etapas de análise adicionais, então seguidas de etapas de modelagem de projetos para representar a classe de projeto elaborada (os detalhes do componente).

O projeto de componentes se inicia neste ponto. Os detalhes do componente **ServiçoDeImpressão** devem ser elaborados para fornecer informações suficientes para orientar a implementação. A classe de análise original é elaborada para dar corpo a todos os atributos e operações necessárias para implementar a classe na forma do componente **ServiçoDeImpressão**. Referindo-se à parte inferior direita da Figura 10.1, a classe de projeto elaborada, **ServiçoDeImpressão**, contém informações de atributos mais detalhadas, bem como uma descrição expandida das operações necessárias para implementar o componente. As interfaces `calcularServiço` e `iniciarServiço` implicam comunicação e colaboração com outros componentes (não indicados aqui). Por exemplo, a operação `calcularCustoDaPágina()` (parte da interface `cálculoDeServiço`) poderia colaborar com um componente **TabelaDePreço** contendo informações sobre os preços de serviços. A operação `verificarPrioridade()` (parte da interface `inícioDeServiço`) poderia colaborar com um componente **FilaDeServiço** para determinar os tipos e prioridades dos serviços atualmente em espera para produção.

A atividade de elaboração é aplicada a todos os componentes definidos como parte do projeto da arquitetura. Uma vez completada, aplica-se uma maior elaboração a cada atributo, operação e interface. As estruturas de dados apropriadas para cada atributo devem ser especificadas. Além disso, o detalhe algorítmico exigido para implementar a lógica de processamento associada a cada operação é desenvolvido. Essa atividade de projeto procedural é discutida posteriormente, ainda neste capítulo. Por fim, desenvolvem-se os mecanismos necessários para implementar a

interface. Para software orientado a objetos, isso poderia englobar a descrição de todas as mensagens necessárias para efetivar a comunicação entre objetos de um sistema.

10.1.2 A visão tradicional

No contexto da engenharia de software tradicional, componente é o elemento funcional de um programa que incorpora a lógica de processamento, as estruturas de dados internas necessárias para implementar a lógica de processamento e uma interface que habilita o componente a ser chamado e que dados sejam passados a ele. Um componente tradicional, também denominado *módulo*, reside na arquitetura de software e se presta a um de três importantes papéis: (1) um *componente de controle* que coordena a chamada de todos os demais componentes do domínio do problema, (2) um *componente de domínio do problema* que implementa uma função completa ou parcial solicitada pelo cliente ou (3) um *componente de infraestrutura* responsável por funções que dão suporte ao processamento necessário no domínio do problema.

Assim como os componentes orientados a objetos, os componentes de software tradicionais são obtidos do modelo de análise. Nesse caso, entretanto, o elemento orientado a fluxos de dados do modelo de análise serve como base para essa obtenção. Cada transformação (bolha), representada nos níveis mais baixos do diagrama de fluxo de dados, é mapeada em uma hierarquia de módulos. Os componentes de controle (módulos) residem próximo do alto da hierarquia (arquitetura de programas), e os componentes de domínio do problema tendem a residir mais próximo da parte inferior da hierarquia. Para alcançar modularidade efetiva, conceitos de projeto como independência funcional (Capítulo 8) são aplicados durante a elaboração dos componentes.

Para ilustrarmos o processo de elaboração de projetos para componentes tradicionais, consideremos novamente o software a ser criado para uma gráfica sofisticada. Um conjunto de diagramas de fluxo de dados seria obtido durante a modelagem de requisitos. Suponhamos que estas sejam mapeadas na arquitetura da Figura 10.2. Cada retângulo representa um componente de software. Observe que os retângulos sombreados são equivalentes, em termos funcionais, às operações definidas para a classe **ServiçoDeImpressão** discutidas na Seção 10.1.1. Nesse caso, entretanto, cada operação é representada como um módulo separado chamado conforme indicado na figura. São usados outros módulos para controlar o processamento, e estes são, portanto, componentes de controle.

Durante o projeto de componentes, cada módulo da Figura 10.2 é elaborado. A interface de módulos é definida explicitamente. Ou seja, cada objeto de dados ou de controle que flui através da interface é representado. São definidas as estruturas de dados utilizadas internamente ao módulo. O algoritmo que possibilita ao módulo cumprir sua função é desenhado usando-se o método de refinamento gradual discutido no Capítulo 8. Algumas vezes o comportamento do módulo é representado usando-se um diagrama de estado.

Para ilustrarmos o processo, consideremos o módulo *CalcularCustoPorPágina*. O intuito é calcular o custo de impressão por página tomando como base as especificações fornecidas pelo cliente. Os dados necessários para realizar essa função são: o **número de páginas contidas no documento**, o **número total de documentos a ser produzidos**, **impressão frente ou frente e verso**, **requisitos de cores**, **requisitos de tamanho**. Esses dados são passados para *CalcularCustoPorPágina* através da interface do módulo. *CalcularCustoPorPágina* usa-os para determinar o custo de uma página baseado no tamanho e na complexidade do trabalho — uma função de todos os dados passados para o módulo via interface. O custo por página é inversamente proporcional ao tamanho do serviço e diretamente proporcional à complexidade do serviço.

A Figura 10.3 representa o projeto de componentes usando uma notação UML modificada. O módulo *CalcularCustoPorPágina* acessa dados chamando o módulo *obterDadosDoServiço*, que possibilita que todos os dados relevantes sejam passados para o componente e uma interface de banco de dados, *acessarBDCustos*, que possibilita que o módulo acesse um banco de dados contendo todos os custos de impressão. À medida que o projeto continua, o módulo

“Invariavelmente, constata-se que um sistema complexo que funciona é a evolução de um sistema simples que funcionava.”

John Gall



À medida que o projeto para cada componente de software é elaborado, o foco passa para o projeto de estruturas de dados específicas e projeto procedural para manipular as estruturas de dados. Entretanto, não se esqueça da arquitetura que deve abrigar os componentes ou as estruturas de dados globais que poderão atender vários componentes.

FIGURA 10.2

Diagrama de estruturas para um sistema tradicional

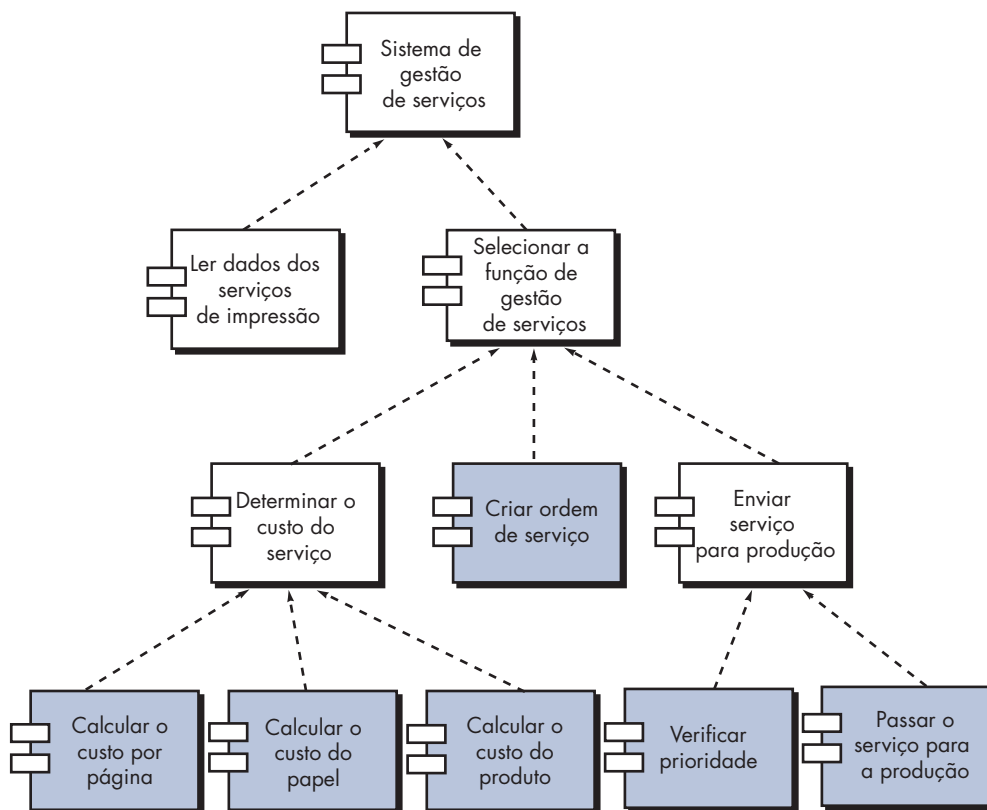
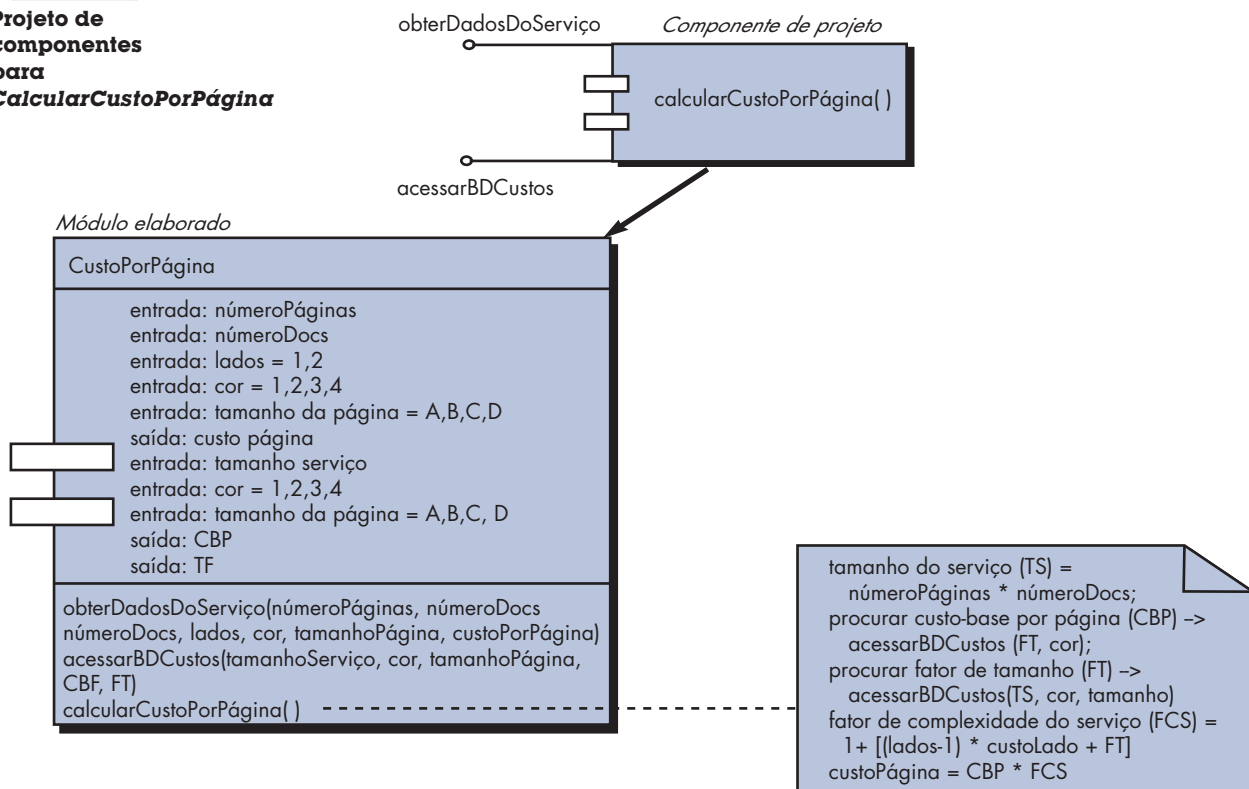


FIGURA 10.3

Projeto de componentes para *CalcularCustoPorPágina*



CalcularCustoPorPágina é elaborado para fornecer detalhes algorítmicos e detalhes de interface (Figura 10.3). Os detalhes algorítmicos podem ser representados usando-se o texto em pseudocódigo mostrado na figura ou por meio de um diagrama de atividades UML. As interfaces são representadas como um conjunto de itens ou objetos de dados de entrada/saída. A elaboração de projeto continua até que detalhes suficientes tenham sido fornecidos para orientar a construção do componente.

10.1.3 Uma visão relacionada com processos

As visões tradicionais e orientadas a objetos de projeto de componentes apresentadas nas Seções 10.1.1 e 10.1.2 partem do pressuposto de que o componente está sendo desenhado a partir do zero. Isto é, temos de criar um componente baseado nas especificações obtidas do modelo de requisitos. Existe, obviamente, uma outra abordagem. Ao longo das duas últimas décadas, a comunidade de engenharia de software tem enfatizado a necessidade de se construírem sistemas que façam uso de componentes de software ou de padrões de projeto existentes. Em essência, é colocado à disposição dos profissionais de software um catálogo de projetos ou código de componentes de qualidade comprovada à medida que o trabalho de projeto prossegue. À medida que a arquitetura de software é desenvolvida, escolhemos componentes ou padrões de projeto desse catálogo e os usamos para preencher a arquitetura. Pelo fato de esses componentes terem sido criados tendo a reusabilidade em mente, uma descrição completa de suas interfaces, a(s) função(ões) por eles realizada(s) e a comunicação e colaboração por eles exigidas estarão à nossa disposição. Discutiremos posteriormente, na Seção 10.6, alguns dos importantes aspectos da engenharia de software baseada em componentes (*component-based software engineering*, CBSE).



Padrões e estruturas baseadas em componentes

Um dos elementos-chave que levam ao sucesso ou insucesso da CBSE é a disponibilidade de padrões baseados em componentes, algumas vezes denominados *middleware*. *Middleware* é um conjunto de componentes de infraestrutura que possibilita que os componentes do domínio do problema se comuniquem entre si através de uma rede ou em um sistema complexo. Os engenheiros de software que quiserem usar desenvolvimento baseado em componentes como processo de software poderão escolhê-los entre os seguintes padrões:

OMG CORBA — www.corba.org/

Microsoft COM — www.microsoft.com/com/tech/complus.asp

Microsoft .NET — <http://msdn2.microsoft.com/en-us/netframework/default.aspx>

Sun JavaBeans — <http://java.sun.com/products/ejb/>

Os sites citados apresentam uma ampla gama de tutoriais, artigos, ferramentas e recursos gerais sobre os importantes padrões de *middleware*.

INFORMAÇÕES

10.2 PROJETO DE COMPONENTES BASEADOS EM CLASSES

Conforme já citado, o projeto de componentes apoia-se nas informações desenvolvidas como parte do modelo de requisitos (Capítulos 6 e 7) e representadas como parte do modelo da arquitetura (Capítulo 9). Quando é escolhida uma abordagem de engenharia de software orientada a objetos, o projeto de componentes focaliza a elaboração de classes específicas do domínio do problema e a definição e o refinamento de classes de infraestrutura contidas no modelo de requisitos. A descrição detalhada dos atributos, operações e interfaces utilizados por essas classes é o detalhe de projeto requerido como precursor da atividade de construção.

10.2.1 Princípios básicos de projeto

Quatro princípios básicos de projeto são aplicáveis ao projeto de componentes e têm sido largamente adotados quando se aplica a engenharia de software orientada a objetos. A motivação por trás da aplicação desses princípios é criar projetos que sejam mais fáceis de modificar

e reduzir a propagação de efeitos colaterais na ocorrência de modificações. Podemos usar tais princípios como guia à medida que cada componente de software é desenvolvido.

Princípio do aberto-fechado (Open-Closed Principle, OCP). “Um módulo [componente] deve ser aberto para a extensão, mas fechado para modificações” [Mar00]. Essa afirmação pode parecer uma contradição, mas representa uma das características mais importantes de um bom projeto de componentes. Em outras palavras, devemos especificar o componente para permitir que ele seja estendido (em seu domínio funcional) sem a necessidade de fazer modificações internas (em nível de código ou lógica) no próprio componente. Para tanto, criamos abstrações que servem como um buffer entre a funcionalidade que provavelmente será estendida e a classe de projeto em si.

Por exemplo, suponhamos que a função de segurança do *CasaSegura* faça uso de uma classe **Detector** que deve verificar o estado de cada tipo de sensor de segurança. É provável que à medida que o tempo for passando, o número e os tipos de sensores de segurança crescerão. Se a lógica de processamento interna for implementada como uma sequência de construtos *se-então-senão*, cada um deles tratando de um tipo de sensor diferente, a adição de um novo tipo de sensor exigirá lógica de processamento interna adicional (ainda um outro *se-então-senão*). Essa é uma violação do OCP.

Uma forma de concretizar o OCP para a classe **Detector** é ilustrada na Figura 10.4. A interface *sensor* apresenta uma visão consistente dos sensores para o componente detector. Se for adicionado um novo tipo de sensor, nenhuma mudança é necessária na classe **Detector** (componente). O OCP é preservado.

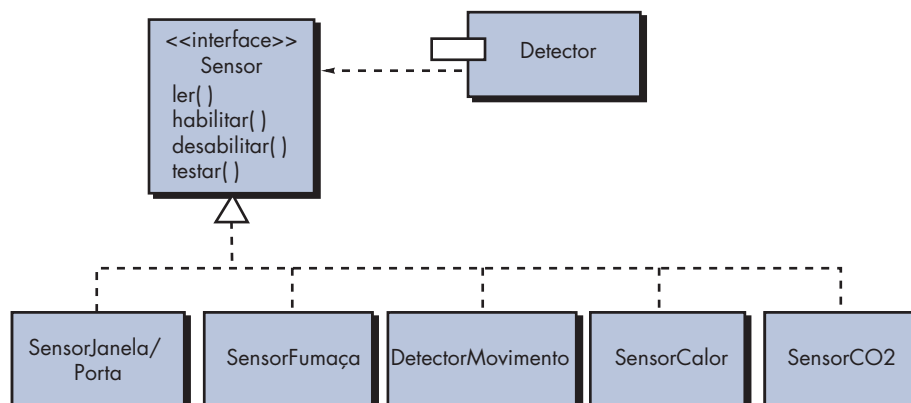
O princípio da substituição de Liskov (LSP). “As subclasses devem ser substitutas de suas classes-base” [Mar00]. Esse princípio de projeto, originalmente proposto por Barbara Liskov [Lis88], sugere que um componente que usa uma classe-base deveria continuar a funcionar apropriadamente caso uma classe derivada da classe-base fosse passada para o componente em seu lugar. O LSP exige que qualquer classe derivada de uma classe-base deva honrar qualquer contrato implícito entre a classe-base e os componentes que a usam. No contexto dessa discussão, um “contrato” é uma *precondição* que deve ser verdadeira antes de o componente usar uma classe-base e uma *pós-condição* que deve ser verdadeira após o componente usar uma classe-base. Ao criar classes derivadas, certifique-se de que atendem às pré e pós-condições.

Princípio da inversão da dependência (DIP). “Dependa de abstrações. Não dependa de concretizações” [Mar00]. Como vimos na discussão sobre OCP, abstrações é o lugar onde um projeto pode ser estendido sem grandes complicações. Quanto mais um componente depender de outros componentes concretos (e não de abstrações como uma interface), mais difícil será estendê-lo.



Se você dispensa o projeto e vai direto ao código, lembre-se apenas de que código é a “concretização” final. Você estará violando o DIP.

FIGURA 10.4
Seguindo o OCP



CASA SEGURA



O OCP em ação

Cena: Sala do Vinod.

Atores: Vinod e Shakira — membros da equipe de engenharia de software do CasaSegura.

Conversa:

Vinod: Acabei de receber uma ligação do Doug [o gerente da equipe]. Ele me disse que o pessoal de marketing quer acrescentar um novo sensor.

Shakira (com um sorriso de superioridade): Outra vez não!

Vinod: Isso mesmo... E você não vai acreditar no que esses caras inventaram.

Shakira: Qual é a surpresa agora?

Vinod (sorrindo): Algo que eles chamaram de sensor de angústia de uau-uau.

Shakira: O quê... ?

Vinod: Destina-se a pessoas que deixam seus bichinhos de estimação em apartamentos ou condomínios ou casas que são próximas uma das outras. O cachorro começa a latir. O vizinho se irrita e reclama. Com esse sensor, se o cachorro latir por, digamos, mais de um minuto, o sensor ativa um modo de alarme especial que chama o dono no seu celular.

Shakira: Você tá brincando, não é?

Vinod: Não. Doug quer saber quanto tempo levará para acrescentar essa característica à função de segurança.

Shakira (pensando um pouco): Não muito... Veja. [Ela mostra ao Vinod a Figura 10.4] Isolamos as verdadeiras classes de sensores por trás da interface **sensor**. Desde que tenhamos as especificações para o sensor de cachorros, acrescentá-lo deve ser moleza. A única coisa que terei de fazer é criar um componente apropriado... Quer dizer, classe, para ele. Nenhuma mudança no componente **Detector** em si.

Vinod: Então, vou dizer ao Doug que não é nada do outro mundo.

Shakira: Conhecendo o Doug, ele nos manterá focados e não irá liberar essa coisa até a próxima versão.

Vinod: Isso não é ruim, mas você consegue implementá-lo agora se ele quiser?

Shakira: Sim, o jeito que projetamos a interface me permite fazê-lo sem grandes complicações.

Vinod (pensando um pouco): Você já ouviu falar do princípio do aberto-fechado?

Shakira (encolhendo os ombros.): Nunca ouvi falar.

Vinod (sorrindo): Sem problemas.

Princípio da segregação de interfaces (ISP). *“É melhor usar várias interfaces específicas de clientes do que uma única interface de propósito geral”* [Mar00]. Há várias ocasiões em que componentes para vários clientes usam uma operação fornecida por uma classe-servidora. O ISP sugere a criação de uma interface especializada para atender cada categoria principal de clientes. Apenas as operações que forem relevantes a determinada categoria de clientes devem ser especificadas na interface para esse cliente. Se vários clientes precisarem das mesmas operações, estas devem ser especificadas em cada uma das interfaces especializadas.

Como um exemplo, considere a classe **Planta** usada para as funções de segurança e vigilância do *ClasseSegura* (Capítulo 6). Para as funções de segurança, **Planta** é usada apenas durante atividades de configuração e usa as operações *colocarDispositivo()*, *mostrarDispositivo()*, *agruparDispositivo()* e *removerDispositivo()* para inserir, mostrar, agrupar e remover sensores da planta. A função de vigilância do *CasaSegura* usa as quatro operações citadas para segurança, porém, também requer operações especiais para gerenciar câmeras: *mostrarFOV()* e *mostrarIDDispositivo()*. Portanto, o ISP sugere que componentes clientes das duas funções do *CasaSegura* tenham interfaces especializadas definidas para eles. A interface para segurança englobaria apenas as operações *colocarDispositivo()*, *mostrarDispositivo()*, *agruparDispositivo()* e *removerDispositivo()*. A interface para vigilância incorporaria as operações *colocarDispositivo()*, *mostrarDispositivo()*, *agruparDispositivo()* e *removerDispositivo()*, juntamente com *mostrarFOV()* e *mostrarIDDispositivo()*.

Embora os princípios de projeto de componentes sejam úteis em termos de orientação, os componentes em si não vivem de forma isolada. Em muitos casos, componentes ou classes individuais são organizados em subsistemas ou pacotes. É razoável perguntar como deve ocorrer a atividade de empacotamento. Exatamente de que forma os componentes devem ser organizados à medida que o projeto prossegue? Martin [Mar00] sugere outros princípios de empacotamento aplicáveis ao projeto de componentes:

PONTO-CHAVE

Projetar componentes para reutilização requer mais que um bom projeto técnico. Essa atividade também requer mecanismos de controle de configuração efetivos (Capítulo 22).

Princípio da equivalência de reuso de versões (REP). “A granularidade da reutilização é a granularidade da versão” [Mar00]. Quando as classes ou os componentes são projetados tendo em vista o reuso, existe um contrato implícito estabelecido entre o desenvolvedor da entidade reutilizável e quem irá usá-la. O desenvolvedor se compromete a estabelecer um sistema de controle de versões que ofereça suporte e manutenção a versões mais antigas da entidade enquanto os usuários gradualmente irão atualizando para a versão mais atual. Em vez de tratar cada uma dessas classes individualmente, em geral é recomendável agrupar classes reutilizáveis em pacotes que possam ser gerenciados e controlados à medida que versões mais recentes evoluam.

Princípio do fechamento comum (CCP). “Classes que mudam juntas, devem ficar juntas” [Mar00]. As classes devem ser empacotadas de forma coesa. Ao serem empacotadas como parte de um projeto, devem tratar a mesma área funcional ou comportamental. Quando alguma característica dessa área tiver de mudar, é provável que apenas aquelas classes contidas no pacote precisarão ser modificadas. Isso leva a um controle de mudanças e gerenciamento de versões mais efetivo.

Princípio comum da reutilização (CRP). “As classes que não são reutilizadas juntas não devem ser agrupadas juntas” [Mar00]. Quando uma ou mais classes contidas em um pacote muda(m), o número da versão do pacote muda. Todas as demais classes ou pacotes que dependem do pacote alterado agora precisam ser atualizadas para a versão mais recente do pacote e serem testadas para garantir que a nova versão opere sem incidentes. Se as classes não forem agrupadas de forma coesa, é possível que uma classe sem nenhuma relação com as demais contidas em um pacote seja alterada. Isso precipitará integração e testes desnecessários. Por essa razão, apenas as classes reutilizadas juntas devem ser incluídas em um pacote.

10.2.2 Diretrizes para o projeto de componentes

Além dos princípios discutidos na Seção 10.2.1, pode-se aplicar um conjunto de diretrizes de projeto pragmáticas à medida que o projeto de componentes prossegue. Tais diretrizes se aplicam a componentes, suas interfaces e às características de dependência e herança que têm algum impacto sobre o projeto resultante. Ambler [Amb02b] sugere as seguintes diretrizes:

Componentes. Devem-se estabelecer convenções de nomes para componentes especificados como parte do modelo de arquitetura e, então, refiná-los e elaborá-los como parte do modelo de componentes. Os nomes de componentes de arquitetura devem ser extraídos do domínio do problema e devem ter significado para todos os interessados que veem o modelo da arquitetura. Por exemplo, o nome de classe **Planta** é significativo para qualquer um que o leia, independentemente de sua bagagem técnica. Por outro lado, componentes de infraestrutura ou classes elaboradas no nível de componentes devem receber nomes que reflitam significados específicos à implementação. Se uma lista ligada tiver de ser gerenciada como parte da implementação **Planta**, a operação *gerenciarLista()* é apropriada, mesmo que uma pessoa não técnica possa interpretá-la de forma incorreta.³ Podemos optar pelo uso de estereótipos para auxiliar na identificação da natureza dos componentes no nível de projeto mais detalhado. Por exemplo, poderíamos usar `<<infrastructure>>` para identificar um componente de infraestrutura, `<<database>>` para identificar um banco de dados que atenda uma ou mais classes de projeto ou o sistema inteiro; `<<table>>` poderia ser usado para identificar uma tabela em um banco de dados.

Interfaces. As interfaces nos dão importantes informações sobre a comunicação e a colaboração (bem como nos ajuda a alcançar o OCP). Entretanto, a representação totalmente livre de interfaces tende a complicar os diagramas de componentes. Ambler [Amb02c] recomenda o seguinte: (1) a representação “pirulito” de uma interface deve ser usada em conjunto com a abordagem mais formal da UML que usa retângulos e setas pontilhadas, quando os diagramas se tornarem mais complexos; (2) para consistência, as interfaces devem fluir da esquerda para a direita do

 O que devemos considerar ao atribuímos nomes aos componentes?

³ É pouco provável que alguém de marketing ou da empresa do cliente (uma pessoa não técnica) examine informações detalhadas de projeto.

retângulo de componente; (3) devem ser mostradas apenas aquelas interfaces relevantes para o componente em consideração, mesmo que estejam disponíveis outras interfaces. Tais recomendações destinam-se a simplificar a natureza visual dos diagramas de componentes da UML.

Dependências e herança. Para melhorar a legibilidade, é aconselhável modelar as dependências da esquerda para a direita e as heranças de baixo (classes derivadas) para cima (classes-base). Além disso, as interdependências de componentes devem ser representadas através de interfaces, e não por meio da representação de uma dependência componente-para-componente. Seguindo a filosofia do OCP, isso facilitará a manutenção do sistema.

10.2.3 Coesão

No Capítulo 8, descrevemos coesão como o “foco único” de um componente. No contexto do projeto de componentes para sistemas orientados a objetos, *coesão* implica um componente ou classe encapsular apenas atributos e operações que estejam intimamente relacionados entre si e com a classe ou o componente em si. Lethbridge e Laganière [Let01] definem uma série de tipos diferentes de coesão (enumerados em ordem de nível de coesão⁴):

Funcional. Apresentada basicamente por operações, este nível de coesão ocorre quando um componente realiza um cálculo planejado e depois retorna um resultado.

De camadas. Apresentada por pacotes, componentes e classes, esse tipo de coesão ocorre quando uma camada mais alta acessa os serviços de uma camada mais baixa, porém as camadas mais baixas não acessam as mais elevadas. Consideremos, por exemplo, que a função de segurança do *CasaSegura* precise fazer uma ligação telefônica caso um alarme seja acionado. Seria possível definirmos um conjunto de pacotes em camadas conforme mostra a Figura 10.5. Os pacotes sombreados contêm componentes de infraestrutura. O acesso é do pacote de painel de controle para baixo.

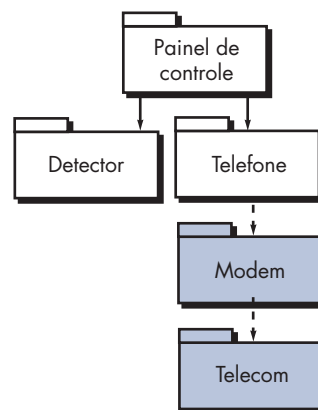
De comunicação. Todas as operações que acessam os mesmos dados são definidas em uma classe. Em geral, tais classes enfocam exclusivamente os dados em questão, acessando-os e armazenando-os.

As classes e componentes que apresentam coesão funcional, de camadas e comunicação são relativamente fáceis de ser implementados, testados e mantidos. Devemos nos esforçar ao máximo para atingir esses níveis de coesão sempre que possível. É importante notar, entretanto, que questões pragmáticas de projeto e implementação algumas vezes nos forçam a optar por níveis de coesão mais baixos.



Embora a compreensão dos vários níveis de coesão seja instrutiva, é mais importante estar atento ao conceito geral ao se projetarem componentes. Mantenha o nível de coesão o mais elevado possível.

FIGURA 10.5
Coesão de camadas



⁴ Em geral, quanto maior o nível de coesão, mais fácil é implementar, testar e manter um componente.

CASASEGURA

**Coesão em ação**

Cena: Sala do Jamie.

Atores: Jamie e Ed — membros da equipe de engenharia de software do CasaSegura que estão trabalhando na função de vigilância.

Conversa:

Ed: Tenho um projeto preliminar do componente **câmera**.

Jamie: Quer fazer uma rápida revisão?

Ed: Creio que sim... Mas, na realidade, gostaria de sua opinião sobre algo.

(Jamie gesticula para que ele continue.)

Ed: Originalmente definimos cinco operações para **câmera**.

Veja...

`determinarTipo()` informa o tipo de câmera.

`traduzirLocalização()` possibilita que a câmera seja movimentada pela planta.

`exibirID()` obtém o ID da câmera e o exibe próximo do ícone câmera.

`exibirVisão()` mostra graficamente o campo de visão da câmera.

`exibirZoom()` mostra a ampliação da câmera graficamente.

Ed: Projetei cada uma delas separadamente e são operações bastante simples. Portanto, imagino que seria uma boa ideia combinar todas as operações de exibição em apenas uma chamada `exibirCâmera()` — ela mostrará o ID, a vista e a ampliação. O que você acha?

Jamie (fazendo uma careta): Não tenho certeza de que seja uma ideia tão boa assim.

Ed (franzindo a testa): O motivo é que todas essas pequenas operações poderão nos dar dor de cabeça.

Jamie: O problema em combiná-las é perdermos coesão, sabe, a operação `exibirCâmera()` não será focada.

Ed (ligeiramente exasperado): E daí? O conjunto todo terá menos de 100 linhas de código-fonte, no máximo. Será mais fácil implementá-lo, eu acho.

Jamie: E se o Marketing decidir mudar a maneira pela qual representamos o campo de visão?

Ed: Simplesmente mexo na operação `displayCâmera()` e faço a modificação.

Jamie: E os efeitos colaterais?

Ed: O que você quer dizer com isso?

Jamie: Bem, digamos que você faça a modificação, mas, inadvertidamente, crie um problema na exibição do ID.

Ed: Eu não seria tão descuidado assim.

Jamie: Talvez não, mas e se alguém do suporte tiver que, daqui a dois anos, fazer a modificação. Pode ser que não entenda a operação tão bem quanto você e, vai saber, talvez ele seja descuidado.

Ed: Então você é contrário a isso?

Jamie: Você é o projetista... A decisão é sua... Apenas certifique-se de ter compreendido as consequências da baixa coesão.

Ed (refletindo um pouco): Talvez seja melhor mesmo fazer duas operações de exibição separadas.

Jamie: Ótima decisão.

10.2.4 Acoplamento

À medida que o projeto para cada componente de software é elaborado, o foco passa para o projeto de estruturas de dados específicas e para o projeto procedural para a manipulação de estruturas de dados. Entretanto, não se esqueça da arquitetura que deve abrigar os componentes ou as estruturas de dados globais que podem servir vários componentes.

Em discussões anteriores sobre análise e projeto, observamos que a comunicação e a colaboração são elementos essenciais de qualquer sistema orientado a objetos. Há, entretanto, o lado sinistro dessa importante (e necessária) característica. Como o volume de comunicação e colaboração aumenta (isto é, à medida que o grau de “conexão” entre as classes aumenta), a complexidade do sistema também cresce. E à medida que a complexidade aumenta, a dificuldade de implementação, testes e manutenção do software também aumentam.

O **acoplamento** é uma medida qualitativa do grau com que as classes estão ligadas entre si. Conforme as classes (e os componentes) se tornam mais interdependentes, o acoplamento aumenta. Um objetivo importante no projeto de componentes é manter o acoplamento o mais baixo possível.

O acoplamento de classes pode se manifestar de uma série de formas. Lethbridge e Laganière [Let01] definem as seguintes categorias de acoplamento:

Acoplamento por conteúdo. Ocorre quando um componente “modifica de forma sub-reptícia os dados internos a um outro componente” [Let01]. Isso viola o encapsulamento — um conceito de projeto básico.

Acoplamento comum. Ocorre quando uma série de componentes faz uso de uma variável global. Embora algumas vezes isso seja necessário (por exemplo, para estabelecer valores-padrão aplicáveis em uma aplicação), o acoplamento comum pode levar à propagação de erros incontrolada e efeitos colaterais não previstos quando forem feitas modificações.

Acoplamento por controle. Ocorre quando a operação *A()* chama a operação *B()* e passa um flag de controle para *B*. O flag de controle “dirige” então a lógica de fluxo no interior de *B*. O problema com essa forma de acoplamento é que uma mudança não relacionada em *B* pode resultar na necessidade de alterar o significado do flag de controle que *A* passa. Se isso for menosprezado, acontecerá um erro.

Acoplamento “carimbo”. Ocorre quando **ClasseB** é declarada como um tipo para um argumento de uma operação da **ClasseA**. Como agora **ClasseB** faz parte da definição da **ClasseA**, modificar o sistema torna-se mais complexo.

Acoplamento por dados. Ocorre quando operações passam longas strings como argumentos de dados. A “largura de banda” da comunicação entre classes e componentes aumenta e a complexidade da interface cresce. Os testes e a manutenção são mais difíceis.

Acoplamento por chamadas de rotinas. Ocorre quando uma operação chama outra. Esse nível de acoplamento é comum e, quase sempre, necessário. Entretanto, realmente aumenta a conectividade de um sistema.

Acoplamento por uso de tipos. Ocorre quando o componente **A** usa um tipo de dados definido em um componente **B** (por exemplo, isso ocorre toda vez que “uma classe declarar uma variável de instância ou uma variável local como tendo outra classe para seu tipo” [Let01]). Se a definição de tipo mudar, todo componente que usa a definição também tem de ser alterado.

Acoplamento por inclusão ou importação. Ocorre quando um componente **A** importa ou inclui um pacote ou o conteúdo do componente **B**.

Acoplamento externo. Ocorre quando um componente se comunica ou colabora com componentes de infraestrutura (por exemplo, funções do sistema operacional, capacidade de bancos de dados, funções de telecomunicação). Embora esse tipo de acoplamento seja necessário, deve se limitar a um pequeno número de componentes ou classes em um sistema.

CASA SEGURA



Acoplamento em ação

Cena: Sala de Shakira.

Atores: Vinod e Shakira — membros da equipe de engenharia de software do CasaSegura que estão trabalhando na função de segurança.

Conversa:

Shakira: Pensei que tive uma grande ideia... Depois refleti um pouco mais a respeito e me pareceu não ser uma ideia tão boa assim. Por fim, a rejeitei, mas pensei que deveria apresentá-la a você.

Vinod: Certamente. Qual é a ideia?

Shakira: Bem, cada um dos sensores reconhece uma condição de alarme de algum tipo, certo?

Vinod (sorrindo): É por isso que os chamamos de sensores, Shakira.

Shakira (exasperada): Sarcasmo, Vinod, você tem que trabalhar suas habilidades interpessoais.

Vinod: Voltando ao que você dizia...

Shakira: Certo, de qualquer modo, imaginei... Por que não criar uma operação em cada objeto sensor chamada fazerCha-

mada() que colaboraria diretamente com o componente **ChamadaExterna**, enfim, com uma interface para o componente **ChamadaExterna**.

Vinod (pensativo): Você quer dizer que em vez de fazer com que a colaboração ocorra fora de um componente como **PainelControle** ou algo do gênero?

Shakira: Exato... Mas depois pensei que cada objeto sensor será associado ao componente **ChamadaExterna** e que isso significa que ele está indiretamente acoplado ao mundo exterior e... Bem, simplesmente imaginei que ele tornaria as coisas mais complicadas.

Vinod: Concordo. Nesse caso, é melhor deixar que a interface de sensores passe informações para o **PainelControle** e deixe que ele inicie a chamada telefônica. Além disso, diferentes sensores talvez resultem em diferentes números de telefone. Você não vai querer que o sensor armazene essas informações... Se elas mudarem...

Shakira: Parece que isso não está certo.

Vinod: A heurística de projeto para acoplamento nos diz que não é correto.

Shakira: Que seja...

Um software deve se comunicar interna e externamente. Consequentemente, acoplamento é uma realidade a ser enfrentada. Entretanto, o projetista deve se esforçar para reduzir o acoplamento sempre que possível e compreender as ramificações do acoplamento elevado quando não puder ser evitado.

10.3 CONDUÇÃO DE PROJETOS DE COMPONENTES

“Se eu tivesse mais tempo, teria escrito uma carta mais curta.”

Blaise Pascal



Se estiver trabalhando em um ambiente não orientado a objetos, as três primeiras etapas concentram-se no refinamento de objetos de dados e funções de processamento (transformações) identificadas como parte do modelo de requisitos.

No início deste capítulo citei que o projeto de componentes é de natureza elaborada. Temos de transformar informações de modelos de arquitetura e requisitos em uma representação de projeto que nos dê detalhes suficientes para orientar a atividade da construção (codificação e testes). As etapas a seguir representam um conjunto de tarefas típico para um projeto de componentes, quando ele é aplicado a um sistema orientado a objetos.

Etapla 1. Identificar todas as classes de projeto correspondentes ao domínio do problema. Usando o modelo de requisitos e de arquitetura, cada classe de análise e componente de arquitetura é elaborada conforme descrito na Seção 10.1.1.

Etapla 2. Identificar todas as classes de projeto correspondentes ao domínio de infraestrutura. Essas classes não são descritas no modelo de requisitos e normalmente não estão presentes no modelo de arquitetura; porém, têm de ser descritas nesse ponto. Conforme já dito, entre as classes e componentes dessa categoria temos os componentes de interfaces gráficas do usuário (muitas vezes disponíveis na forma de componentes reutilizáveis), em componentes de sistemas operacionais, bem como em componentes de administração de dados e objetos.

Etapla 3. Elaborar todas as classes de projeto que não são obtidas como componentes reutilizáveis. A elaboração requer que todas as interfaces, atributos e operações necessários para implementar a classe sejam descritos em detalhe. A heurística de projeto (por exemplo, coesão e acoplamento de componentes) deve ser considerada à medida que essa tarefa é conduzida.

Etapla 3a. Especificar detalhes de mensagens quando classes ou componentes colaboram entre si. O modelo de requisitos faz uso de um diagrama de colaboração para mostrar como as classes de análise colaboram entre si. À medida que o projeto de componentes prossegue, algumas vezes é útil mostrar os detalhes dessas colaborações especificando a estrutura das mensagens passadas entre objetos de um sistema. Embora essa atividade de projeto seja opcional, pode ser utilizada como um precursor da especificação de interfaces que mostram como os componentes em um sistema se comunicam e colaboram entre si.

A Figura 10.6 ilustra um diagrama de colaboração simples para o sistema de impressão discutido anteriormente. Três objetos, **ServiçoDeProdução**, **OrdemDeServiço** e **FilaDeServiço**, colaboram na preparação de um serviço de impressão a ser submetido ao fluxo de produção. São passadas mensagens entre os objetos conforme ilustrado pelas setas da figura. Durante a modelagem de requisitos as mensagens são especificadas conforme mostra a figura. Entretanto, à medida que o projeto prossegue, cada mensagem é elaborada por meio da expansão de sua sintaxe, da seguinte maneira [Ben02]:

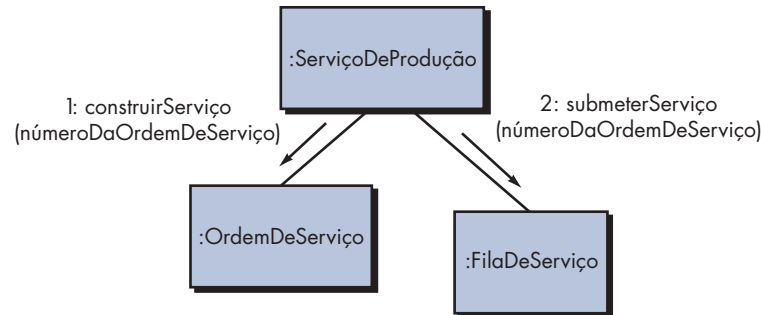
[condição de guarda] sequência de expressões (valor de retorno) :=
nome da mensagem (lista de argumentos)

em que uma [condição de guarda] é escrita em linguagem de restrição de objetos (*Object Constraint Language*, OCL)⁵ e que especifica qualquer conjunto de condições que devem ser atendidas antes de a mensagem poder ser enviada; **sequência de expressões** é um valor inteiro (ou outro indicador de ordem, por exemplo, 3.1.2) que indica a ordem sequencial em que uma mensagem é enviada; (**valor de retorno**) é o nome das informações retornadas por uma operação

⁵ A OCL é discutida brevemente no Apêndice 1.

FIGURA 10.6

Diagrama de colaboração com as mensagens



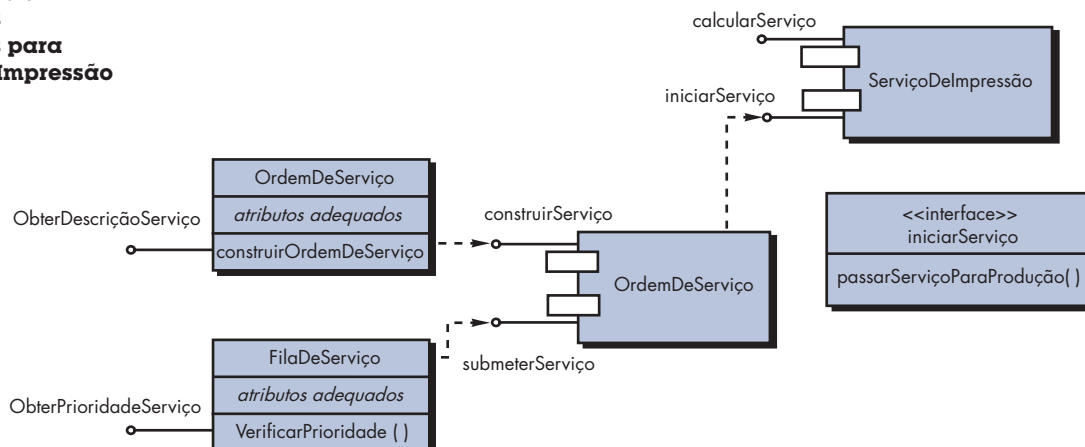
chamada pela mensagem; **nome da mensagem** identifica uma operação a ser chamada e (**lista de argumentos**) é a lista de atributos passados para a operação.

Etapa 3b. Identificar interfaces adequadas para cada componente. No contexto de projeto de componentes, uma interface UML é “um grupo de operações externamente visíveis (públicas). A interface não contém nenhuma estrutura interna, nenhum atributo, nenhuma associação...” [Ben02]. Colocado mais formalmente, interface equivale a uma classe abstrata que fornece uma conexão controlada entre as classes de projeto. A elaboração de interfaces é ilustrada na Figura 10.1. Em essência, operações definidas para o diagrama de classes são classificadas em uma ou mais classes abstratas. Todas as operações contidas em uma classe abstrata (a interface) devem ser coesas; elas devem apresentar processamento focado em uma função ou subfunção limitada.

Referindo-se à Figura 10.1, pode ser alegado que a interface *iniciarServiço* não apresenta coesão suficiente. Na realidade, ela realiza três subfunções distintas — criar uma ordem de serviço, verificar a prioridade do serviço e passar um serviço para produção. O projeto de interfaces deve ser refatorado. Uma abordagem poderia ser reexaminar as classes de projeto e definir uma nova classe **OrdemDeServiço** que cuidaria de todas as atividades associadas à montagem de uma ordem de serviço. A operação *construirOrdemDeServiço()* passa a fazer parte dessa classe. De modo similar, poderíamos definir uma classe **FilaDeServiço** que incorporaria a operação *verificarPrioridade()*. Uma classe **ServiçoDeProdução** englobaria todas as informações associadas a um serviço de produção a ser passado para o centro de produção. A interface *iniciarServiço* assumiria então a forma indicada na Figura 10.7. A interface *iniciarServiço* agora é coesa,

FIGURA 10.7

Interfaces de refatoração e definições de classes para ServiçoDeImpressão



focalizando uma função. As interfaces associadas a **ServiçoDeProdução**, **OrdemDeServiço** e **FilaDeServiço** são similarmente coesas.

Etapla 3c. Elaborar atributos e definir tipos de dados e estruturas de dados necessárias para implementá-los. Em geral, as estruturas e tipos de dados utilizados para definir atributos são definidas no contexto da linguagem de programação que será usada para implementação. A UML define o tipo de dados de um atributo usando a seguinte sintaxe:

nome : **tipo da expressão** = **valor inicial** {**string de propriedades**}

em que **nome** é o nome do atributo, **tipo da expressão** é o tipo de dados, **valor inicial** é o valor que o atributo assume quando um objeto é criado e **string de propriedades** define uma propriedade ou característica do atributo.

Durante a primeira iteração de projeto de componentes, os atributos normalmente são descritos por nomes. Referindo-se mais uma vez à Figura 10.1, a lista de atributos para **ServiçoDeImpressão** enumera apenas os nomes dos atributos. Entretanto, à medida que a elaboração do projeto prossegue, cada atributo é definido usando o formato de atributo UML citado. Por exemplo, **peso-TipoDoPapel** é definido da seguinte maneira:

peso-TipoDoPapel: **string** = "A" { contém 1 de 4 valores - A, B, C ou D }

que define **peso-TipoDoPapel** como uma variável de **string** inicializada para o valor A que pode assumir qualquer um dos quatro valores do conjunto {A,B,C, D}.

Se um atributo aparecer repetidamente ao longo de uma série de classes de projeto e tiver uma estrutura relativamente complexa, é melhor criar uma classe separada para acomodá-lo.

Etapla 3d. Descrever detalhadamente o fluxo de processamento contido em cada operação. Isso poderia ser concretizado usando-se um pseudocódigo baseado em linguagem de programação ou através de um diagrama de atividades UML. Cada componente de software é elaborado por meio de uma série de iterações que aplicam o conceito de refinamento gradual (Capítulo 8).

A primeira iteração define cada operação como parte da classe de projeto. Em cada caso, a operação deve ser caracterizada de modo a garantir alta coesão; a operação deve realizar uma única função ou subfunção determinada. A iteração seguinte faz pouco mais que expandir o nome da operação. Por exemplo, a operação *calcularCustoPapel()* indicada na Figura 10.1 pode ser expandida da seguinte maneira:

calcularCustoPapel (**peso, tamanho, cor**): **numérico**

Isso indica que *calcularCustoPapel ()* requer os atributos **peso**, **tamanho** e **cor** como entrada e retorna um valor numérico (na verdade um valor monetário) como saída.

Se o algoritmo necessário para implementar *calcularCustoPapel ()* for simples e largamente compreendido, talvez não seja necessário maior elaboração de projeto. O engenheiro de software que realiza a codificação fornece os detalhes necessários para implementar a operação. Entretanto, se o algoritmo for mais complexo ou enigmático, será necessária maior elaboração de projeto nesse estágio. A Figura 10.8 representa um diagrama de atividades UML para *calcularCustoPapel ()*. Quando os diagramas de atividades são usados para a especificação de projeto de componentes, em geral são representados em um nível de abstração ligeiramente maior que o do código-fonte. Uma abordagem alternativa — o uso de pseudocódigo para especificação de projeto — é discutida na Seção 10.5.3.

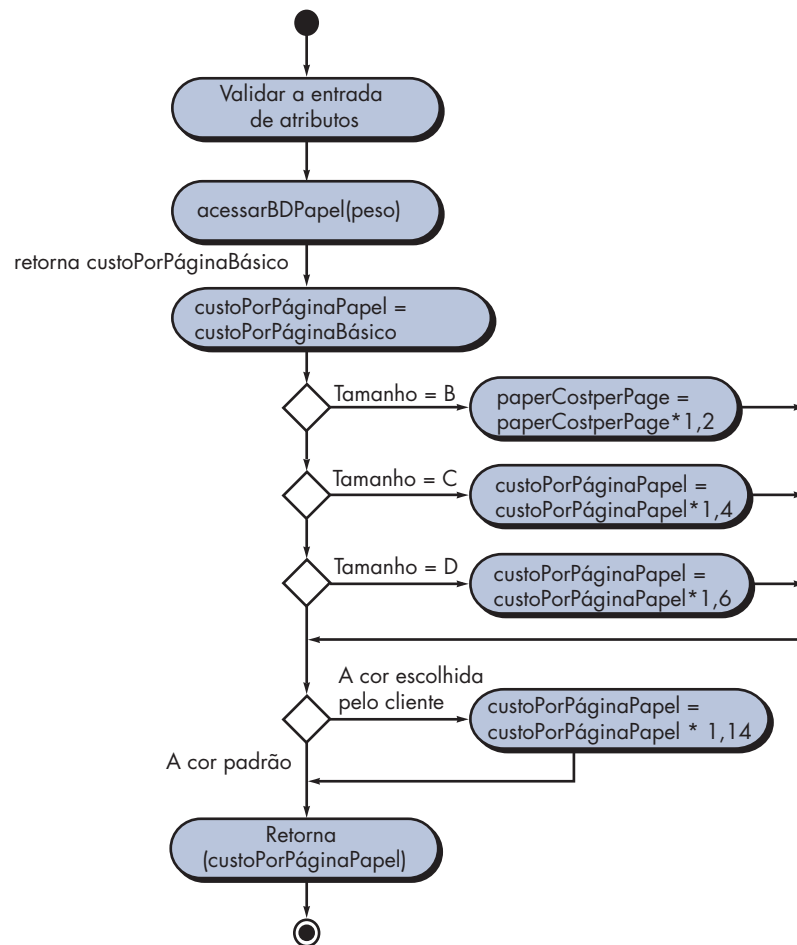
Etapla 4. Descrever fontes de dados persistentes (bancos de dados e arquivos) e identificar as classes necessárias para gerenciá-los. Os bancos de dados e arquivos normalmente transcendem a descrição de projeto de um componente individual. Na maioria dos casos, esses repositórios de dados persistentes são especificados inicialmente como parte do projeto da arquitetura. Entretanto, à medida que a elaboração do projeto



Use elaboração gradual à medida que for refinando o projeto de componentes. Pergunte sempre "Existe uma maneira pela qual isso possa ser simplificado e ainda assim obter o mesmo resultado?"

FIGURA 10.8

Diagrama de atividades UML para `calcularCustoPapel()`



prossegue, é útil fornecer detalhes adicionais sobre a estrutura e a organização das fontes de dados persistentes.

Etapla 5. Desenvolver e elaborar representações comportamentais para uma classe ou componente.

Usaram-se diagramas de estados UML como parte do modelo de requisitos para representar o comportamento externamente observável do sistema e o comportamento mais localizado de classes de análise individuais. Durante o projeto de componentes, algumas vezes é necessário modelar o comportamento de uma classe de projeto.

O comportamento dinâmico de um objeto (uma instanciãção de uma classe de projeto à medida que o programa é executado) é afetado por eventos externos a ele e pelo estado atual (modo de comportamento) do objeto. Para compreender o comportamento dinâmico de um objeto, você deve examinar todos os casos de uso relevantes para a classe de projeto ao longo de sua vida. Estes fornecem informações que o ajudam a delinear os eventos que afetam o objeto e os estados em que o objeto reside à medida que o tempo passa e os eventos ocorrem. As transições entre estados (dirigidos por eventos) são representadas usando-se um diagrama de estado UML [Ben02], conforme ilustrado na Figura 10.9.

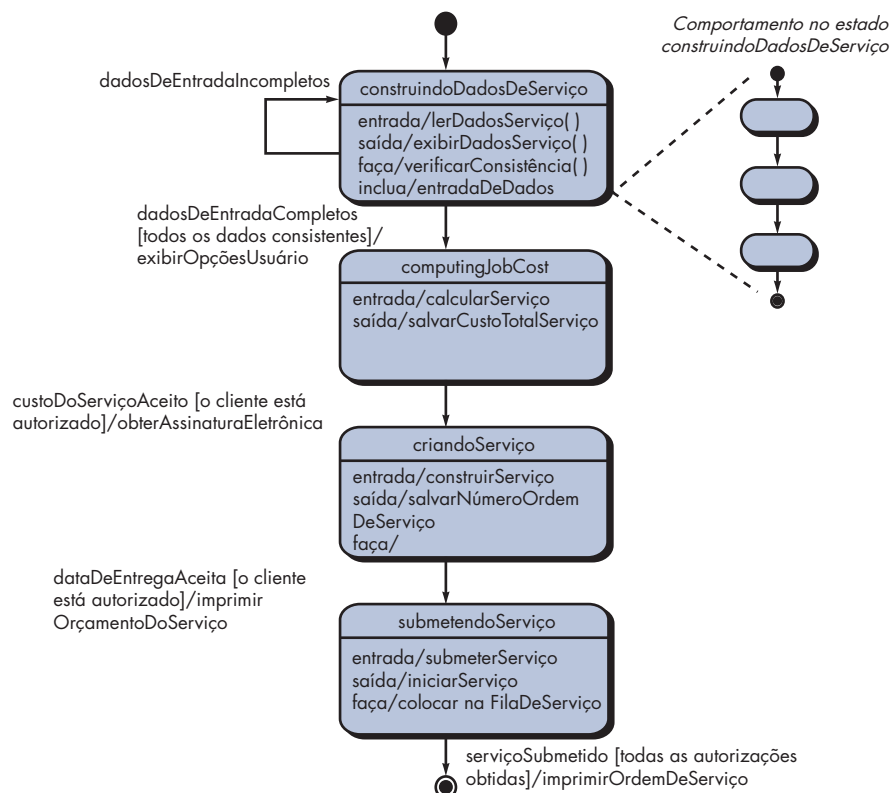
A transição de um estado (representado por um retângulo com cantos arredondados) para outro ocorre como consequência de um evento que assume a seguinte forma:

nome-evento (**lista-parâmetros**) [**condição-de-guarda**] / **expressão de ação**

em que **nome-evento** identifica o evento, **lista-parâmetros** incorpora dados associados ao evento, **condição-de-guarda** é escrito em OCL e especifica uma condição que deve ser atendida antes de o

FIGURA 10.9

Fragmento de um diagrama de estados para a classe **ServiçoDeImpressão**



evento poder ocorrer, e **expressão de ação** define uma ação que ocorre à medida que a transição ocorre.

Referindo-se à Figura 10.9, cada estado poderia definir ações *entrada/* e *saída/* que acontecem, respectivamente, à medida que ocorre a transição para o estado e fora dele. Na maioria dos casos, essas ações correspondem a operações relevantes à classe que está sendo modelada. O indicador *faça/* fornece um mecanismo para indicar atividades que ocorrem enquanto se encontram em determinado estado, ao passo que o indicador *inclua/* fornece um meio para elaborar o comportamento através da incorporação de mais detalhes de diagramas de estados na definição de um estado.

É importante notar que o modelo comportamental contém informações que não são óbvias em outros modelos de projeto. Por exemplo, o exame cuidadoso dos diagramas de estado da Figura 10.9 indica que o comportamento dinâmico da classe **ServiçoDeImpressão** é dependente de duas aprovações do cliente à medida que dados de custo e cronograma para o serviço de impressão são obtidos. Sem as aprovações (a condição de controle garante que o cliente é autorizado para aprovar), o serviço de impressão não pode ser submetido, pois não há nenhuma maneira de atingir o estado *submetendoServiço*.

Etapla 6. Elaborar diagramas de implantação para fornecer detalhes de implementação adicionais. Os diagramas de implantação (Capítulo 8) são usados como parte do projeto da arquitetura e representados na forma de descritores. Dessa forma, funções importantes do sistema (representadas como subsistemas) são representadas no contexto do ambiente computacional que irá abrigá-los.

Durante o projeto de componentes, os diagramas de implantação podem ser elaborados para representar a localização de pacotes de componentes fundamentais. Entretanto, em geral os componentes não são representados individualmente em um diagrama de componentes. A razão para tal é evitar a complexidade dos diagramas. Em alguns casos, os diagramas de

implantação são elaborados na forma de instância naquele momento. Isso significa que o(s) ambiente(s) de sistema operacional e de hardware específicos utilizados é(são) especificado(s) e a localização de pacotes de componentes nesse ambiente é indicada.

Etapa 7. Refatorar toda representação de projetos de componentes e sempre considerar alternativas. Ao longo deste livro, tenho enfatizado que projeto é um processo iterativo. O primeiro modelo no nível de componentes que criamos não será tão completo, consistente ou acurado quanto o da *enésima* iteração aplicada ao modelo. É essencial refatorar à medida que o trabalho de projeto é conduzido.

Além disso, não se deve ter uma visão restrita. Sempre há soluções alternativas de projeto, e os melhores projetistas consideram todas (ou quase todas) elas antes de se decidirem pelo modelo de projetos final. Desenvolva alternativas e considere cuidadosamente cada uma delas, usando os princípios e conceitos apresentados no Capítulo 8 e neste capítulo.

10.4 PROJETO DE COMPONENTES PARA WEBAPPS

A fronteira entre conteúdo e função normalmente fica obscura quando se consideram sistemas e aplicações baseadas na Web (WebApps). Consequentemente, é razoável perguntar: O que é um componente WebApp?

No contexto do presente capítulo, um componente de WebApp é (1) uma função coesa bem definida que manipula conteúdo ou fornece processamento computacional ou de dados para um usuário final ou (2) um pacote coeso de conteúdo e funcionalidade que forneça ao usuário final alguma capacidade exigida. Consequentemente, o projeto de componentes para WebApps em geral incorpora elementos de projeto de conteúdo e de projeto funcional.

10.4.1 Projeto de conteúdo para componentes

O projeto de conteúdo para componentes concentra-se nos objetos de conteúdo e na maneira como poderiam ser empacotados para apresentação a um usuário final de uma WebApp. Consideremos, por exemplo, um recurso de vigilância de vídeo baseado na Web contido em **CasaSeguraGarantida.com**. Entre os diversos recursos, o usuário pode escolher e controlar qualquer uma das câmeras representadas como parte da planta, requisitar imagens em miniaturas capturadas por vídeo de todas as câmeras e exibir vídeos *streaming* de qualquer uma das câmeras. Além disso, o usuário poderá controlar o deslocamento e a ampliação de uma câmera usando ícones de controle apropriados.

Podemos definir uma série de componentes de conteúdo potenciais para o recurso de vigilância de vídeo: (1) os objetos de conteúdo que representam a distribuição de espaço (a planta) com ícones adicionais indicando a posição de sensores e câmeras de vídeo, (2) o conjunto de imagens em miniaturas capturadas por vídeo (cada um deles um objeto de dados distinto) e (3) a janela de vídeo *streaming* para uma câmera específica. A cada um dos componentes pode ser atribuído um nome e estes podem ser manipulados como um único pacote.

Consideremos uma planta que represente quatro câmeras estrategicamente colocadas no interior de uma casa. Por meio da solicitação do usuário, é capturado um quadro de cada câmera e estes identificados como um objeto de conteúdo gerado dinamicamente, **CapturaDeVídeoN**, em que *N* identifica as câmeras 1 a 4. Um componente de conteúdo, chamado **ImagensEm-Miniatura**, combina todos os quatro objetos de conteúdo **CapturaDeVídeoN** e os exibe na página de vigilância de vídeo.

A formalidade do projeto de conteúdo para componentes deve ser ajustada às características da WebApp a ser construída. Em muitos casos, os objetos de conteúdo não precisam ser organizados como componentes e podem ser manipulados individualmente. Entretanto, à medida que o tamanho e a complexidade (da WebApp, dos objetos de conteúdo e suas inter-relações) forem crescendo, talvez seja necessário organizar o conteúdo para permitir uma manipulação de pro-

jeto e referência mais fácil.⁶ Além disso, se o conteúdo for altamente dinâmico (por exemplo, o conteúdo para um site de leilões on-line), é importante estabelecer um claro modelo estrutural que incorpore os componentes de conteúdo.

10.4.2 Projeto funcional para componentes

As aplicações Web modernas oferecem funções de processamento cada vez mais sofisticadas que (1) realizam processamento localizado para gerar capacidades de conteúdo e navegação de maneira dinâmica, (2) fornecem recursos de processamento de dados ou cálculos apropriados para a área de aplicação das WebApps, (3) fornecem acesso e consultas sofisticadas a bancos de dados ou (4) estabelecem interfaces de dados com sistemas corporativos externos. Para conseguirmos esses (e muitos outros) recursos, projetamos e construímos componentes funcionais para WebApps que sejam similares a componentes de software para software convencional.

A funcionalidade da WebApp é fornecida através de uma série de componentes desenvolvidos em paralelo com o projeto das informações para garantir que sejam consistentes. Em essência, partimos da consideração do modelo de requisitos, bem como da arquitetura das informações iniciais e, em seguida, examinamos como a funcionalidade afeta a interação do usuário com a aplicação, as informações que são apresentadas e as tarefas de usuário realizadas.

Durante o projeto da arquitetura, o conteúdo e a funcionalidade da WebApp são combinados para criar uma *arquitetura funcional*, que é uma representação do domínio funcional da WebApp e descreve os componentes funcionais fundamentais da WebApp e como interagem entre si.

Por exemplo, as funções de deslocamento e ampliação de imagem para o recurso de vigilância por vídeo do **CasaSeguraGarantida.com** são implementadas como parte de um componente **ControleDeCâmera**. Como alternativa, o deslocamento e a ampliação de imagem podem ser implementados como operações, *deslocar()* e *ampliar()*, que fazem parte da classe **Câmera**. Em ambos os casos, a funcionalidade implicada pelas funções de deslocamento e ampliação de imagem deve ser implementada como módulos contidos em **CasaSeguraGarantida.com**.

10.5 PROJETO DE COMPONENTES TRADICIONAIS

PONTO-CHAVE

Programação estruturada é uma técnica de projeto que restringe o fluxo lógico a três construções: sequência, condição e repetição.

Os fundamentos para o projeto de componentes para componentes de software tradicionais⁷ foram formados no início dos anos 1960 e foram solidificados com o trabalho de Edsger Dijkstra e seus colegas ([Boh66], [Dij65], [Dij76b]). No final dos anos 1960, Dijkstra e outros propuseram o uso de um conjunto de construções lógicas restritas a partir das quais qualquer programa poderia ser formado. As construções enfatizavam “a manutenção do domínio funcional”. Ou seja, cada construção possuía uma estrutura lógica previsível e entra-se nela pela parte superior e sai-se pela inferior, possibilitando a um leitor seguir mais facilmente o fluxo procedural.

As construções são sequência, condição e repetição. *Sequência* implementa etapas de processamento essenciais na especificação de qualquer algoritmo. *Condição* fornece a facilidade para processamento seletivo baseado em alguma ocorrência lógica e *repetição* possibilita o looping. Essas três construções são fundamentais para a *programação estruturada* — uma importante técnica para projetos de componentes.

As construções estruturadas foram propostas para limitar o projeto procedural de software a um pequeno número de estruturas lógicas previsíveis. As métricas de complexidade (Capítulo 23) indicam que o uso de construções estruturadas reduz a complexidade dos programas e, consequentemente, facilitam a legibilidade, a realização de testes e manutenção. O uso de um número

⁶ Os componentes de conteúdo também podem ser reutilizados em outras WebApps.

⁷ Um componente de software tradicional implementa um elemento de processamento que trata uma função ou subfunção no domínio do problema ou alguma capacidade no domínio da infraestrutura. Em geral denominados módulos, procedimentos ou sub-rotinas, os componentes tradicionais não encapsulam dados da mesma forma que os componentes orientados a objetos.

limitado de construções lógicas também contribui para o processo de compreensão humana chamado pelos psicólogos de *agrupamento*. Para compreender esse processo, considere a maneira como você está lendo esta página. Não lemos as letras individualmente, mas reconhecemos padrões ou grupos de letras que formam as palavras ou frases. As construções estruturadas são grupos lógicos que permitem a um leitor reconhecer elementos procedurais de um módulo, em vez de ter de ler o projeto ou código linha por linha. A compreensão aumenta quando são encontrados padrões lógicos reconhecíveis.

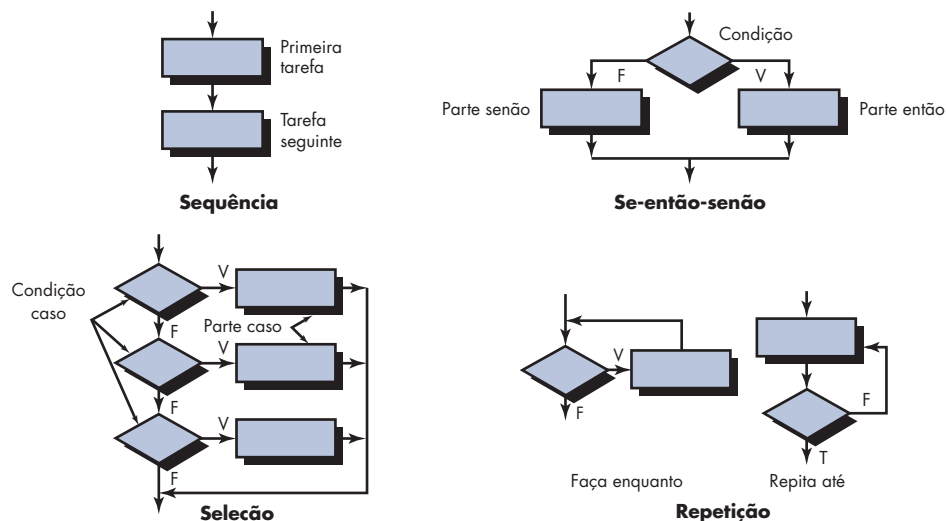
Qualquer programa, independentemente da área de aplicação ou de sua complexidade técnica, pode ser projetado e implementado usando-se apenas as três construções estruturadas. Entretanto, deve-se frisar que o uso dogmático apenas dessas construções pode, algumas vezes, provocar dificuldades práticas. A Seção 10.5.1 considera a questão de forma mais detalhada.

10.5.1 Notação gráfica em projeto

“Uma figura vale mais do que mil palavras”, porém é muito importante saber qual figura e quais mil palavras. Não há nenhuma dúvida de que ferramentas gráficas, como o fluxograma ou o diagrama de atividades UML, dispõem de padrões pictóricos muito úteis que prontamente representam detalhes procedurais. Entretanto, se as ferramentas gráficas forem mal utilizadas, a figura errada poderá levar a um software errado.

O diagrama de atividades permite que representemos sequência, condição e repetição — todos elementos da programação estruturada — e é um descendente de uma representação pictórica de projetos anterior (ainda largamente utilizada) chamada *fluxograma*, que assim como o diagrama de atividades, é pictoricamente bastante simples. É usado um retângulo para indicar uma etapa de processamento. Um losango representa uma condição lógica e setas mostram o fluxo de controle. A Figura 10.10 ilustra três construções estruturadas. A *sequência* é representada na forma de dois retângulos de processamento conectados por meio de uma linha (seta) de controle. Uma *condição*, também chamada *se-então-senão*, é representada por um losango de decisão que, se verdadeira, faz com que o processamento da *parte então* ocorra e, se falsa, chama o processamento da *parte senão*. Uma *repetição* é representada usando-se duas formas ligeiramente diferentes. O *faça enquanto* testa uma condição e executa uma tarefa em loop repetitivamente até que a condição seja verdadeira. Um *repita até* primeiro executa a tarefa em loop e depois testa a condição e repete a tarefa até que a condição falhe. A construção de *seleção*

FIGURA 10.10
Construções de fluxograma



(ou *seleção de caso*) mostrada na figura é, na verdade, uma extensão de *se-então-senão*. Um parâmetro é testado por decisões sucessivas até a ocorrência de uma condição verdadeira e um caminho de processamento de um caso é executado.

Em geral, o uso dogmático apenas das construções estruturadas pode introduzir ineficiência quando for necessária uma saída de um conjunto de laços aninhados ou de condições aninhadas. Mais importante ainda, a complicação adicional de todos os testes lógicos ao longo do caminho de saída pode confundir o fluxo de controle do software, aumentar a possibilidade de erros e ter um impacto negativo na legibilidade e na facilidade de manutenção. O que fazer então?

Restam duas opções: (1) a representação procedural ser redesenhada de modo que a “ramificação de saída” não seja necessária em um ponto aninhado no fluxo de controle ou (2) as construções estruturadas serem violadas de uma maneira controlada; ou seja, é criada uma ramificação com restrição para fora do fluxo aninhado. A opção 1 é, obviamente, a abordagem ideal, porém a opção 2 pode ser aceita sem violar o espírito da programação estruturada.

10.5.2 Notação tabular de projeto



Use uma tabela de decisão quando for encontrado um conjunto complexo de condições e ações em um componente.

Em diversas aplicações de software, pode ser necessário um módulo para avaliar uma combinação complexa de condições e selecionar ações apropriadas com base nessas condições. As *tabelas de decisão* [Hur83] dispõem de uma notação que transforma ações e condições (descritas em uma narrativa de processamento ou caso de uso) em forma tabular. A tabela é difícil de ser mal interpretada e até poderia ser usada como entrada legível por máquina para um algoritmo dirigido por tabelas.

A organização de uma tabela de decisão é ilustrada na Figura 10.11. Na figura, a tabela é dividida em quatro seções. O quadrante superior esquerdo contém uma lista de todas as condições. O quadrante inferior esquerdo contém uma lista de todas as ações possíveis com base em combinações de condições. Os quadrantes à direita formam uma matriz que indica combinações de condição e as ações correspondentes que ocorrerão a determinada combinação. Consequentemente, cada coluna da matriz pode ser interpretada como uma *regra de processamento*. As etapas a seguir são aplicadas para desenvolver uma tabela de decisão:



Como construir uma tabela de decisões?

1. Listar todas as ações que podem ser associadas a um procedimento (ou componente) específico.
2. Listar todas as condições (ou decisões tomadas) durante a execução do procedimento.
3. Associar conjuntos de condições específicos a ações específicas, eliminando combinações de condições impossíveis; como alternativa, desenvolver toda permutação de condições possível.
4. Definir regras por meio da indicação de quais ações ocorrem para um conjunto de condições.

Para ilustrarmos o emprego de uma tabela de decisão, consideremos o seguinte exemplo de um caso de uso informal que acaba de ser proposto para o sistema de gráfica:

São definidos três tipos de clientes: um cliente comum, um cliente prata e um cliente ouro (esses tipos são atribuídos conforme o volume de negócios realizado pelo cliente com a gráfica nos últimos 12 meses). Um cliente comum terá preços e prazos de entrega normais. Um cliente prata obtém um desconto de 8% em todos os orçamentos e é colocado à frente de todos os clientes comuns na fila de serviços. Um cliente ouro tem uma redução de 15% nos preços cotados e é colocado à frente de todos os clientes comuns e prata na fila de serviços. Um desconto especial de x por cento, além de outros descontos, poderá ser aplicado aos orçamentos de qualquer cliente a critério da gerência.

A Figura 10.11 ilustra a representação de uma tabela de decisão do caso de uso informal anterior. Cada uma das seis regras indica uma das seis condições viáveis. Como regra geral, a

FIGURA 10.11

**Nomenclatura de
Tabelas de decisão**

Condições	Regras					
	1	2	3	4	5	6
Cliente comum	V	V				
Cliente prata			V	V		
Cliente ouro					V	V
Desconto especial	F	V	F	V	F	V
Ações						
Nenhum desconto	✓					
Aplicar desconto de 8%			✓	✓		
Aplicar desconto de 15%					✓	✓
Aplicar desconto adicional de x%		✓		✓		✓

tabela de decisão pode ser usada efetivamente para complementar outras notações procedurais de projeto.

10.5.3 Linguagem de projeto de programas

A *linguagem de projeto de programas* (*Program Design Language*, PDL), também chamada *inglês estruturado* ou *pseudocódigo*, incorpora a estrutura lógica de uma linguagem de programação com a habilidade expressiva de forma livre de uma linguagem natural (o inglês, por exemplo). Texto narrativo (em inglês, por exemplo) é incorporado à sintaxe de programação parecida com uma língua. Ferramentas automatizadas (por exemplo, [Cai03]) podem ser utilizadas para melhorar a aplicação da PDL.

Uma sintaxe PDL básica deve incluir construções para a definição de componentes, descrição de interfaces, declaração de dados, estruturação de blocos, construções de condição, de repetição e de entrada/saída (I/O). Deve-se notar que a PDL pode ser estendida para incluir palavras-chave para multitarefas e/ou processamento concorrente, tratamento de interrupções, sincronização entre processos e muitas outras características. O projeto de aplicações para a qual a PDL será usada deve ditar a forma final para a linguagem de projeto. O formato e a semântica para algumas dessas construções PDL são apresentados no exemplo a seguir.

Para ilustrarmos o uso da PDL, consideremos um projeto procedural para a função de segurança do *CasaSegura* discutida em capítulos anteriores. O sistema monitora alarmes para incêndio, fumaça, ladrões, água e temperatura (por exemplo, o sistema de aquecimento falha enquanto o proprietário do imóvel encontra-se fora durante o inverno) e produz um alarme sonoro e chama o serviço de monitoramento, gerando uma mensagem de voz sintetizada.

Recorde-se que a PDL *não* é uma linguagem de programação. Pode-se adaptar conforme a necessidade sem a preocupação em relação a erros de sintaxe. Entretanto, o projeto para software de monitoramento teria de ser revisado (você vê algum problema nisso?) e mais refinado antes que o código pudesse ser escrito. As PDLs⁸ a seguir fornecem uma elaboração do projeto procedural para uma versão inicial de um componente de gerenciamento de alarme.

⁸ O nível de detalhe representado pela PDL é definido localmente. Algumas pessoas preferem uma descrição orientada a uma linguagem mais natural, enquanto outros preferem algo mais próximo de código.

```

componente gerenciamentoAlarme;
O intuito deste componente é o de gerenciar chaves de um painel de controle e a entrada dos sensores por
tipo e atuar sob qualquer condição de alarme que seja encontrada.
fixar valores default para
estadosDoSistema (valor de retorno), todos os itens de dados
iniciar todas as portas do sistema e reiniciar todo o hardware
verificar chavesDoPainelDeControle (cpc)
    se cpc = "teste" então iniciar o alarme configurado em "ligado"
    se cpc = "alarmeDesligado" então iniciar o alarme configurado em "desligado"
    se cpc = "novoValorDeLimite" então iniciar o teclado
    se cpc = "alarmeDeAssaltoDesligado" então desativarAlarme
    .
    .
    .
    default para cpc = nada
reiniciar todos os valoresDeSinais e chaves
faça para todos os sensores
    chamar procedimento verificarSensor retornando valorDoSinal
    se valorDoSinal > limite[tipoDeAlarme]
        então mensagemTelefonica = mensagem [tipoAlarme]
        configurar campainhaDoAlarme para "ligada" para tempoAlarmeEmSegundos
        configurar estadoDoSistema = "condiçãoAlarme"
        par-início
            chamar procedimento alarme "ligado", tempoAlarmeEmSegundos
            chamar procedimento telefônico configurado para tipoAlarme, númeroDoTelefone
        par-fim
    senão saia
fim-se
fim faça-para
fim gerenciamentoAlarme

```

Note que o projetista do componente **gerenciamentoAlarme** usou a construção **par-início... par-fim** que especifica um bloco paralelo. Todas as tarefas especificadas no bloco **par-início** são executadas em paralelo. Nesse caso, detalhes de implementação não são considerados.

10.6 DESENVOLVIMENTO BASEADO EM COMPONENTES

No contexto da engenharia de software, a reutilização é uma ideia ao mesmo tempo antiga e nova. Os programadores têm reutilizado ideais, abstrações e processos desde os primórdios da computação, mas a abordagem inicial para a reutilização era *ad hoc*. Hoje em dia, sistemas computacionais complexos e de alta qualidade devem ser construídos em prazos muito curtos e exigem uma abordagem mais organizada para a reutilização.

A engenharia de software baseada em componentes (CBSE) é um processo que enfatiza o projeto e a construção de sistemas baseados em computadores usando “componentes” de software reutilizáveis. Clements [Cle95] descreve a CBSE da seguinte maneira:

A [CBSE] incorpora a filosofia “compre, não construa” defendida por Fred Brooks e outros. Da mesma forma que sub-rotinas iniciais liberavam o programador de pensar em detalhes, a [CBSE] transfere a ênfase de programar software para compor sistemas de software. A implementação deu lugar à integração como foco.

Mas surge uma série de questões. É possível construir sistemas complexos montando-os por meio de um catálogo de componentes de software reutilizáveis? Isso pode ser realizado de maneira efetiva em termos de custo e tempo? Podem ser estabelecidos incentivos apropriados para

“Engenharia de domínio significa encontrar características comuns entre os sistemas para identificar componentes que possam ser aplicados a vários sistemas...”

Paul Clements



O processo de análise discutido nesta seção concentra-se em componentes reutilizáveis. Entretanto, a análise de sistemas comerciais de prateleira completos (por exemplo, aplicações de comércio eletrônico, aplicações para automação da equipe de vendas) também podem fazer parte da análise de domínio.

encorajar os engenheiros de software para a reutilização em vez de reinventar? A gerência está disposta a incorrer na despesa adicional associada à criação de componentes de software reutilizáveis? A biblioteca de componentes necessários para realizar a reutilização pode ser criada para torná-la acessível aos que precisam dela? Os componentes que realmente existem podem ser encontrados por aqueles que precisam deles?

Cada vez mais, a resposta a cada uma dessas questões é “sim”. No restante desta seção, examinaremos algumas questões que devem ser consideradas para tornar a CBSE bem-sucedida em uma organização de engenharia de software.

10.6.1 Engenharia de domínio

O objetivo da *engenharia de domínio* é identificar, construir, catalogar e disseminar um conjunto de componentes de software que tenham aplicabilidade em software existente e futuro em determinado domínio de aplicação.⁹ O objetivo geral é estabelecer mecanismos que permitam aos engenheiros de software compartilhar esses componentes — para reutilizá-los — durante o trabalho em sistemas novos e existentes. A engenharia de *domínio* abrange três atividades principais — análise, construção e disseminação.

A abordagem geral para a *análise de domínio* em geral é caracterizada no contexto da engenharia de software orientada a objetos. As etapas no processo são definidas como:

1. Definir o domínio a ser investigado.
2. Classificar os itens extraídos do domínio.
3. Coletar uma amostra representativa das aplicações do domínio.
4. Analisar cada aplicação na amostra e definir classes de análise.
5. Desenvolver um modelo de requisitos para as classes.

É importante notar que a análise de domínio aplica-se a qualquer paradigma de engenharia de software e poderia ser aplicada tanto em desenvolvimento convencional quanto orientado a objetos.

10.6.2 Qualificação, adaptação e composição de componentes

A engenharia de *domínio* fornece a biblioteca de componentes reutilizáveis necessários para a engenharia de software baseada em componentes. Alguns desses componentes reutilizáveis são desenvolvidos internamente, outros podem ser extraídos de aplicações existentes e outros ainda podem ser adquiridos de terceiros.

Infelizmente, a existência de componentes reutilizáveis não garante que possam ser integrados de forma fácil e efetiva à arquitetura escolhida para uma nova aplicação. É por essa razão que uma sequência de atividades de desenvolvimento baseado em componentes é aplicada quando um componente é proposto para uso.

Qualificação de componentes. A qualificação de componentes garante que um componente candidato irá realizar a função necessária, se ele “se encaixará” adequadamente ao estilo de arquitetura (Capítulo 9) especificada para o sistema e apresentará as características de qualidade (por exemplo, desempenho, confiabilidade, usabilidade) exigidas para a aplicação.

Uma descrição da interface fornece informações úteis sobre a operação e o uso de um componente de software, porém não fornece todas as informações necessárias para determinar se um componente proposto pode, de fato, ser reutilizado efetivamente em nova aplicação. Entre os muitos fatores considerados durante a qualificação de componentes temos [Bro96]:

- Interface de programas aplicativos (*application programming interface*, API).
- Ferramentas de desenvolvimento e integração exigidas pelo componente.

Quais fatores são considerados durante a qualificação de componentes?

⁹ No Capítulo 9 referimo-nos a gêneros de arquitetura que identificam domínios de aplicação específicos.

- Requisitos de tempo de execução, incluindo o emprego de recursos (por exemplo, memória ou armazenamento), timing ou velocidade e protocolo de redes.
- Requisitos de serviços, incluindo interfaces para sistemas operacionais e suporte de outros componentes.
- Características de segurança, incluindo controles de acesso e protocolo de autenticação.
- Pressupostos implícitos de projetos, incluindo o uso de algoritmos numéricos e não numéricos.
- Tratamento de exceções.

Cada um dos fatores é relativamente fácil de avaliar quando são propostos componentes reutilizáveis desenvolvidos internamente. Se forem aplicadas práticas adequadas de engenharia de software durante o desenvolvimento de um componente, poderão ser dadas respostas àquelas questões implícitas na lista. Entretanto, é muito mais difícil determinar o funcionamento interno de componentes comerciais de prateleira (*commercial off-the-shelf*, COTS) ou de terceiros, pois a única informação disponível pode ser a especificação da própria interface.

Adaptação de componentes. Em condições ideais, a engenharia de domínio cria uma biblioteca de componentes que pode ser facilmente integrada à arquitetura de uma aplicação. Para termos a “facilidade de integração” subentende-se que: (1) métodos consistentes de gerenciamento de recursos tenham sido implementados para todos os componentes da biblioteca, (2) atividades comuns como gerenciamento de dados existam para todos os componentes e (3) as interfaces internas da arquitetura e com ambiente externo tenham sido implementadas de maneira consistente.

Na realidade, mesmo após qualificar um componente para uso na arquitetura de uma aplicação, podem ocorrer conflitos em uma ou mais das áreas apenas citadas. Para evitá-los, algumas vezes é usada uma técnica de adaptação chamada *empacotamento de componentes* [Bro96]. Quando uma equipe de software tem acesso completo ao projeto e código internos de um componente (o que normalmente não ocorre a menos que sejam usados componentes comerciais de prateleira com código-fonte aberto), é aplicado um *empacotamento caixa branca*. Da mesma forma que seu equivalente em testes de software (Capítulo 18), o empacotamento caixa branca examina os detalhes de processamento interno do componente e realiza modificações no código para eliminar qualquer conflito. O *empacotamento caixa cinza* é aplicado quando a biblioteca de componentes oferece uma linguagem de extensão de componentes ou uma API que possibilitem a eliminação ou o mascaramento de conflitos. O *empacotamento caixa preta* requer a introdução de pré e pós-processamento na interface de componentes para eliminar ou mascarar conflitos. Temos de determinar se o esforço exigido para empacotar um componente adequadamente se justifica ou se vale mais criar um componente personalizado (projetado para eliminar os conflitos encontrados).

Composição de componentes. A tarefa de composição de componentes monta componentes qualificados, adaptados e construídos para compor a arquitetura estabelecida a uma aplicação. Para tal, deve ser estabelecida uma infraestrutura para vincular os componentes a um sistema operacional. A infraestrutura (em geral uma biblioteca de componentes especializados) fornece um modelo para a coordenação de componentes e serviços específicos que habilitam os componentes a se coordenarem entre si e realizarem tarefas comuns.

Devido ao potencial impacto da reutilização e CBSE no setor de software ser enorme, várias companhias importantes e consórcios da indústria propuseram padrões para software de componentes.¹⁰

OMG/CORBA. O Object Management Group (OMG) publicou uma *arquitetura de agentes de solicitação de objetos comuns* (*common object request broker architecture*, CORBA). Um *agente de solicitação de objetos* (*object request broker*, ORB) oferece uma série de serviços



Além de avaliarmos se o custo da adaptação para a reutilização se justifica, também devemos avaliar se a obtenção da funcionalidade e desempenho exigidos, pode ser feita de modo efetivo em termos de custo.

WebRef

As informações mais recentes sobre CORBA podem ser obtidas em www.omg.org.

¹⁰ Greg Olsen [Ols06] apresenta uma excelente discussão sobre empreendimentos do setor no passado e no presente para tornar a CBSE uma realidade.

que possibilita que componentes (objetos) reutilizáveis se comuniquem com outros componentes, independentemente de sua localização em um sistema.

WebRef

As informações mais recentes sobre JavaBeans podem ser obtidas em java.sun.com/produtos/javabeans/docs/.

Microsoft COM e .NET. A Microsoft desenvolveu um *modelo de objetos de componentes* (*component object model*, COM) que fornece uma especificação para uso de componentes produzidos por vários fabricantes em uma única aplicação que rode no sistema operacional Windows. Do ponto de vista da aplicação, “o foco não é em como [os objetos COM são] implementados, mas sim apenas no fato de que o objeto possui uma interface que ele registra no sistema e que usa o sistema de componentes para se comunicar com outros objetos COM” [Har98a]. A estrutura Microsoft .NET engloba COM e fornece uma biblioteca de classes reutilizáveis que abrange um largo espectro de domínios de aplicação.

WebRef

As informações mais recentes sobre COM e .NET podem ser obtidas em www.microsoft.com/COM e msdn2.microsoft.com/en-us/netframework/default.aspx.

Componentes JavaBeans da Sun. O sistema de componentes JavaBeans é uma infraestrutura CBSE independente de plataforma e portátil desenvolvida com o emprego da linguagem de programação Java. O sistema de componentes JavaBeans engloba um conjunto de ferramentas, denominado BDK (*Bean Development Kit*), que permite aos desenvolvedores: (1) analisar como os Beans (componentes) existentes funcionam, (2) personalizar seus comportamentos e aparências, (3) estabelecer mecanismos para coordenação e comunicação, (4) desenvolver Beans personalizados para uso em uma aplicação específica e (5) testar e avaliar o comportamento do Bean.

Nenhum desses padrões domina o mercado. Embora muitos desenvolvedores tenham adotado como algum deles, é provável que grandes empresas de software optem por adotar um padrão baseado nas plataformas e categorias de aplicação escolhidas.

10.6.3 Análise e projeto para reutilização

Embora o processo de CBSE encoraje o uso de componentes de software existentes, há situações em que componentes de software novos têm de ser desenvolvidos e integrados ao software de prateleira e componentes desenvolvidos internamente já existentes. Pelo fato de novos componentes tornarem-se membros da biblioteca de componentes reutilizáveis desenvolvida dentro da empresa, eles devem ser projetados tendo em vista a reutilização.

Conceitos de projeto como abstração, encapsulamento, independência funcional, refinamento e programação estruturada, juntamente com métodos, testes e garantia da qualidade de software (*software quality assurance*, SQA) orientados a objetos, bem como métodos para verificação da correção (Capítulo 21), todos eles contribuem para a criação de componentes de software reutilizáveis. Nesta subseção, consideraremos questões específicas à reutilização complementares às sólidas práticas de engenharia de software.

O modelo de requisitos é analisado para determinar aqueles elementos que apontam para componentes reutilizáveis existentes. Elementos do modelo de requisitos são comparados com as descrições de componentes reutilizáveis em um processo algumas vezes conhecido como “correspondência da especificação” [Bel95]. Se a correspondência da especificação apontar para um componente existente que atenda às necessidades da aplicação atual, poderemos extrair o componente de uma biblioteca (repositório) de reuso e usá-lo no projeto de um novo sistema. Se os componentes não puderem ser encontrados (se não existir nenhuma correspondência), é criado um novo componente. É nesse ponto — quando se inicia a criação de um novo componente — que o *projeto visando a reutilização* (*design for reuse*, DFR) deve ser considerado.

Conforme já dito, o DFR requer a aplicação de conceitos e princípios de projeto de software sólidos (Capítulo 8). Porém as características do domínio de aplicação também devem ser consideradas. Binder [Bin93] sugere uma série de questões-chave¹¹ que formam a base para o projeto visando a reutilização:

¹¹ Em geral, as preparações DFR devem ser realizadas como parte da engenharia de domínios.



O DFR pode ser bem difícil quando componentes tiverem de ser interfaceados ou integrados com sistemas legados ou com vários sistemas cuja arquitetura e protocolos de interfaceamento são inconsistentes.

Dados padronizados. O domínio de aplicação deve ser investigado e estruturas de dados globais padronizadas (por exemplo, estruturas de arquivos ou um banco de dados completo) devem ser identificadas. Todos os componentes de projeto podem então ser caracterizados para fazer uso dessas estruturas de dados padronizadas.

Protocolos de interface padronizados. Devem ser estabelecidos três níveis de protocolo de interface: a natureza das interfaces intramodulares, o projeto de interfaces técnicas externas (não humanas) e a interface homem-máquina.

Templates de programas. É escolhido um estilo de arquitetura (Capítulo 9) e que pode servir como um template para o projeto da arquitetura do novo software.

Uma vez estabelecidos os templates de programas, de interfaces e de dados, temos uma estrutura na qual criar o projeto. Componentes novos que se adaptem a essa estrutura têm maior probabilidade de posterior reutilização.

10.6.4 Classificação e recuperação de componentes

Consideremos uma grande biblioteca de universidade. Centenas de milhares de livros, periódicos e outras fontes de informação estão disponíveis para uso. Porém, é preciso desenvolver um esquema de classificação e acesso a esses recursos. Para navegar pelo enorme volume de informações, os bibliotecários definiram um esquema de classificação que inclui o código de classificação de Dados Internacionais de Catalogação na Publicação, palavras-chave, nomes de autores e outras entradas de índice. Todos os parâmetros possibilitam ao usuário encontrar um recurso necessário de forma rápida e fácil.

Consideremos agora um grande repositório de componentes. Dezenas de milhares de componentes de software reutilizáveis residem nele. Mas como encontrar aquele que precisamos? Para responder a essa pergunta, surge uma segunda: Como descrever componentes de software em termos inequívocos e classificáveis? Essas são perguntas difíceis e ainda não se chegou a uma resposta definitiva. Na presente seção exploraremos os rumos atuais que permitirão a futuros engenheiros de software navegarem em bibliotecas de reutilização.

Um componente de software reutilizável pode ser descrito de várias maneiras, porém, uma descrição ideal engloba aquilo que Tracz [Tra95] denominou *modelo 3C* — conceito, conteúdo e contexto. O *conceito* de um componente de software é “a descrição daquilo que o componente faz” [Whi95]. A interface para o componente é totalmente descrita e a semântica — representada no contexto das pré e pós-condições — é identificada. O conceito deve comunicar o objetivo do componente. O *conteúdo* de um componente descreve como o conceito é realizado. Em essência, o conteúdo são informações que ficam ocultas para usuários eventuais e que precisam ser conhecidas apenas por aqueles que pretendem modificar ou testar o componente. O *contexto* posiciona um componente de software reutilizável em seu domínio de aplicabilidade. Por meio da especificação de características conceituais, operacionais e de implementação, o contexto possibilita a um engenheiro de software encontrar o componente apropriado para atender os requisitos da aplicação.

Para serem úteis em um ambiente prático, conceito, conteúdo e contexto devem ser traduzidos em um esquema de especificação concreto. Dezenas de trabalhos e artigos foram escritos tratando os esquemas de classificação para componentes de software reutilizáveis (veja, por exemplo, [Cec06] para uma visão geral das tendências atuais).

A classificação nos permite encontrar e recuperar possíveis componentes reutilizáveis, porém o ambiente de reutilização deve existir para integrar efetivamente tais componentes. Um ambiente de reutilização apresenta as seguintes características:

- Um banco de dados de componentes capaz de armazenar componentes de software e as informações de classificação necessárias para recuperá-las.
- Um sistema de gerenciamento de bibliotecas que dê acesso ao banco de dados.

Quais são as principais características de um ambiente de reutilização de componentes?

- Um sistema de recuperação de componentes de software (por exemplo, um agente de solicitação de objetos) que permita a uma aplicação-cliente recuperar componentes e serviços do servidor de bibliotecas.
- As ferramentas CBSE que oferecem suporte à integração de componentes reutilizados em um novo projeto ou implementação.

Cada uma das funções interage com ou está embutida em uma biblioteca de reutilização.

A *biblioteca de reutilização* é um elemento de um repositório de software maior (Capítulo 22) e oferece recursos para o armazenamento de componentes de software e uma ampla gama de artefatos reutilizáveis (por exemplo, especificações, projetos, padrões, estruturas, trechos de código, casos de teste, guias de usuário). A biblioteca abrange um banco de dados e as ferramentas necessárias para consultar o banco de dados e recuperar componentes dele. O esquema de classificação dos componentes serve como base para consultas à biblioteca.

As consultas são em geral caracterizadas usando-se o elemento contexto do modelo 3C descrito anteriormente nesta seção. Se uma consulta inicial resultar em uma lista volumosa de possíveis componentes, a consulta deve ser refinada para restringir a lista. As informações de conceito e conteúdo são então extraídas (depois de possíveis componentes terem sido encontrados) para ajudar na seleção do componente apropriado.

WebRef

Um conjunto completo de recursos sobre CBSE pode ser encontrado em www.cbd-hq.com/.



CBSE

Objetivo: Apoiar a modelagem, projeto, revisão e integração de componentes de software como parte de um sistema mais amplo.

Mecânica: A mecânica das ferramentas varia. Em geral, as ferramentas CBSE auxiliam em uma ou mais das seguintes capacidades: especificação e modelagem da arquitetura de software, navegação e seleção de componentes de software disponíveis; integração de componentes.

Ferramentas representativas¹²

ComponentSource (www.componentsource.com) oferece uma ampla gama de componentes (e ferramentas) de software comerciais suportada em diversos padrões de componentes.

FERRAMENTAS DO SOFTWARE

Component Manager, desenvolvida pela Flashline (www.flashline.com), “é uma aplicação que permite, promove e mede a reutilização de componentes de software”.

Select Component Factory, desenvolvida pela Select Business Solutions (www.selectbs.com), “é um conjunto integrado de produtos para projeto de software, revisão de software,

gerenciamento de serviços/componentes, gerenciamento de requisitos e geração de código”.

Software Through Pictures-ACD, distribuída pela Aonix (www.aonix.com), permite modelagem completa empregando a UML para a arquitetura dirigida a modelos OMG — uma abordagem aberta e desvinculada de qualquer fornecedor específico para CBSE.

10.7 RESUMO

O processo para projeto de componentes abrange uma sequência de atividades que reduz lentamente o nível de abstração com o qual um software é representado. Em última instância, o projeto de componentes representa o software em um nível de abstração próximo do código.

Podem-se adotar três visões diferentes de projeto de componentes, dependendo da natureza do software a ser desenvolvido. A visão orientada a objetos focaliza a elaboração de classes de projeto provenientes tanto do domínio do problema como de infraestrutura. A visão tradicional refina três tipos diferentes de componentes ou módulos: módulos de controle, módulos do domínio do problema e os módulos de infraestrutura. Em ambos os casos, são aplicados conceitos e princípios básicos de projeto que levam a um software de alta qualidade. Quando considerados do ponto de vista de processos, o projeto de componentes faz uso de componentes de software reutilizáveis e padrões de projeto que são elementos fundamentais da engenharia de software baseada em componentes.

¹² As ferramentas aqui apresentadas não significam um aval, mas sim uma amostra dessa categoria. Na maioria dos casos, seus nomes são marcas registradas pelos respectivos desenvolvedores.

Uma série de importantes princípios e conceitos orienta o projetista à medida que as classes são elaboradas. Ideias englobadas pelo princípio do aberto-fechado e da inversão da dependência, além de conceitos como acoplamento e coesão, orientam o engenheiro de software na construção de componentes de software que podem ser testados, implementados e mantidos. Para conduzir projetos de componentes nesse contexto, são elaboradas classes através da especificação de detalhes de mensagens, identificação de interfaces apropriadas, elaboração de atributos e definição de estruturas de dados para implementá-las, descrevendo o fluxo de processamento em cada operação e representando o comportamento em termos de componentes ou classes. Em todos os casos, a iteração (refatoração) de projeto é uma atividade essencial.

O projeto de componentes tradicional requer a representação de estruturas de dados, interfaces e algoritmos para um módulo de programa em detalhes suficientes para orientar-nos na geração de código-fonte em uma linguagem de programação. Para tanto, o projetista usa uma série de notações de projeto que representam detalhes no nível de componentes, em formato gráfico, tabular ou com base em texto.

O projeto de componentes para WebApps considera tanto o conteúdo quanto a funcionalidade, já que é fornecido por um sistema baseado na Web. O projeto de conteúdo no nível de componentes focaliza os objetos de conteúdo e a maneira pela qual podem ser empacotados para apresentação ao usuário final de uma WebApp. O projeto funcional para WebApps concentra-se nas funções de processamento que manipulam conteúdo, efetuam cálculos, consultas e acesso a um banco de dados, bem como estabelecem interfaces com outros sistemas. Todos os princípios e diretrizes de projeto de componentes se aplicam.

Programação estruturada é uma filosofia de projeto procedural que restringe o número e o tipo de construções lógicas usadas para representar detalhes algorítmicos. O intuito da programação estruturada é auxiliar o projetista na definição de algoritmos que sejam menos complexos e, conseqüentemente, mais fáceis de ser lidos, testados e mantidos.

A engenharia de software baseada em componentes identifica, constrói, cataloga e dissemina um conjunto de componentes de software em determinado domínio de aplicação. Esses componentes são então qualificados, adaptados e integrados para uso em um novo sistema. Os componentes reutilizáveis devem ser projetados em ambiente que estabeleça estruturas de dados, protocolos de interface e arquiteturas de programa-padrão para cada domínio de aplicação.

PROBLEMAS E PONTOS A PONDERAR

10.1. O termo *componente* é, algumas vezes, difícil de definir. Forneça inicialmente uma definição genérica e, a seguir, definições mais explícitas para software tradicional e orientado a objetos. Por fim, escolha três linguagens de programação com as quais você esteja familiarizado e ilustre como cada uma define um componente.

10.2. Por que os componentes de controle são necessários em software tradicional e em geral não são em software orientado a objetos?

10.3. Descreva o OCP em suas próprias palavras. Por que é importante criar abstrações que sirvam como interface entre componentes?

10.4. Descreva o DIP em suas próprias palavras. O que poderia acontecer se um projetista criasse uma dependência muito grande nas concretizações?

10.5. Selecione três componentes que você tenha desenvolvido recentemente e avalie os tipos de coesão que cada um apresenta. Caso tivesse de definir o principal benefício da coesão elevada, qual seria?

10.6. Selecione três componentes que você tenha desenvolvido recentemente e avalie os tipos de acoplamento que cada um apresenta. Caso tivesse de definir o principal benefício de um baixo acoplamento, qual seria?

10.7. É razoável dizer que componentes de domínio do problema jamais devem apresentar acoplamento externo? Caso concorde, que tipos de componentes apresentariam acoplamento externo?

10.8. Desenvolva: (1) uma classe de projeto elaborada, (2) descrições de interface, (3) um diagrama de atividades para uma das operações contidas na classe e (4) um diagrama de estados detalhado para uma das classes do *CasaSegura* já discutidas em capítulos anteriores.

10.9. Refinamento gradual e refatoração são a mesma coisa? Em caso negativo, em que diferem?

10.10. O que é um componente de WebApp?

10.11. Selecione um pequeno trecho de um programa existente (aproximadamente 50 a 75 linhas de código-fonte). Isole as construções de programação estruturadas desenhando retângulos em torno delas no código-fonte. O trecho de programa contém construções que violam a filosofia da programação estruturada? Em caso positivo, redesenhe o código para que fique de acordo com as construções de programação estruturadas. Em caso negativo, o que você notou em relação aos retângulos que desenhou?

10.12. Todas as linguagens de programação modernas implementam as construções de programação estruturada. Forneça exemplos de três linguagens de programação.

10.13. Selecione um pequeno componente codificado e represente-o usando: (1) um diagrama de atividades, (2) um fluxograma, (3) uma tabela de decisão e (4) PDL.

10.14. Por que a ideia de “agrupamento” é importante durante o processo de revisão em projetos de componentes?

LEITURAS E FONTES DE INFORMAÇÃO COMPLEMENTARES

Foram publicados muitos livros sobre desenvolvimento baseado em componentes e reutilização de componentes nos últimos anos. Apperly e seus colegas (*Service- and Component-Based Development*, Addison-Wesley, 2003), Heineman e Councill (*Component Based Software Engineering*, Addison-Wesley, 2001), Brown (*Large Scale Component-Based Development*, Prentice-Hall, 2000), Allen (*Realizing e-Business with Components*, Addison-Wesley, 2000), Herzum e Sims (*Business Component Factory*, Wiley, 1999), Allen, Frost e Yourdon (*Component-Based Development for Enterprise Systems: Applying the Select Perspective*, Cambridge University Press, 1998) abordam todos os aspectos importantes do processo de CBSE. Cheesman e Daniels (*UML Components*, Addison-Wesley, 2000) discutem a CBSE com ênfase na UML.

Gao e seus colegas (*Testing and Quality Assurance for Component-Based Software*, Artech House, 2006) e Gross (*Component-Based Software Testing with UML*, Springer, 2005) apresentam questões relacionadas a testes e garantia da qualidade de software para sistemas baseados em componentes.

Chegaram ao mercado dezenas de livros que descrevem os padrões de mercado baseados em componentes nos últimos anos. Estes tratam dos detalhes de funcionamento dos padrões em si, mas também consideram diversos tópicos importantes da CBSE.

O trabalho de Linger, Mills e Witt (*Structured Programming — Theory and Practice*, Addison-Wesley, 1979) se mantém como um verdadeiro tratado sobre o assunto. O texto contém uma ótima PDL, bem como discussões detalhadas sobre as ramificações da programação estruturada. Outros livros que focalizam as questões procedurais para sistemas tradicionais seriam os de Robertson (*Simple Program Design*, 3. ed., Course Technology, 2000), Farrell (*A Guide to Programming Logics and Design*, Course Technology, 1999), Bentley (*Programming Pearls*, 2. ed., Addison-Wesley, 1999) e Dahl (*Structured Programming*, Academic Press, 1997).

Relativamente poucos livros recentes têm se dedicado ao projeto de componentes. Em geral, livros de linguagens de programação tratam o projeto procedural com certo nível de detalhe, mas sempre no contexto da linguagem introduzida pelo livro. Centenas de títulos desse tipo estão disponíveis.

Uma ampla gama de fontes de informação sobre projeto de componentes se encontra disponível na Internet. Uma lista atualizada de referências na Web, relevante para o projeto de componentes, pode ser encontrada no site www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm.